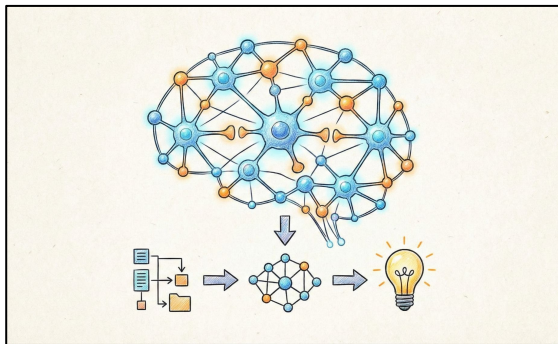


UT3 - Machine Learning y Redes Neuronales



*“De la Programación Explícita
al Aprendizaje Automático”*

¿Qué es realmente el Machine Learning?

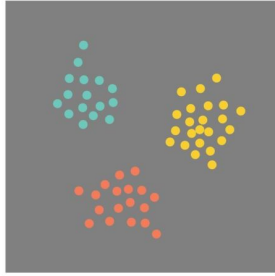
Programación Tradicional: Datos + Reglas $\rightarrow$ Respuestas.

Machine Learning: Datos + Respuestas $\rightarrow$ Reglas (Modelo).

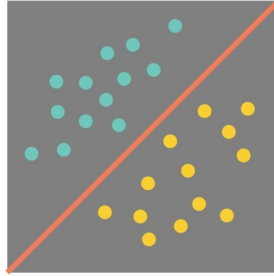
Puntos Clave:

- No programamos la solución, programamos el algoritmo que *encuentra* la solución.
- Capacidad de generalización: El objetivo no es memorizar, es predecir sobre datos nuevos.

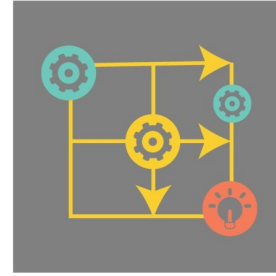
Los 3 Pilares del Machine Learning



Unsupervised
Learning



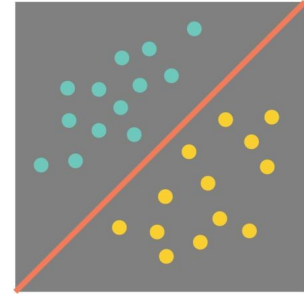
Supervised
Learning



Reinforcement
Learning

Aprendizaje Supervisado

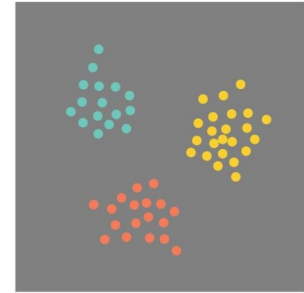
- Tenemos datos etiquetados (Entrada X + Salida y).
- Objetivo: Predecir y para nuevos X .
- Ejemplos: Clasificación (Spam/No Spam), Regresión (Precio de casas).



Supervised
Learning

Aprendizaje No Supervisado

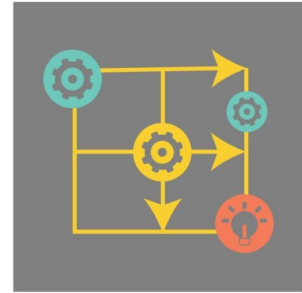
- Datos sin etiquetar (Solo X).
- Objetivo: Encontrar estructuras o patrones ocultos.
- Ejemplos: Clustering (Segmentación de clientes), Reducción de dimensionalidad (PCA).



Unsupervised
Learning

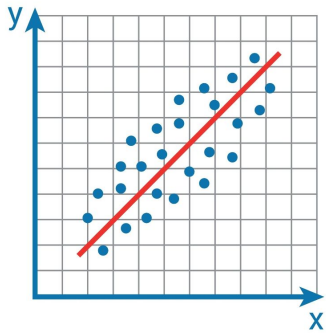
Aprendizaje por Refuerzo

- Agente interactuando con un entorno.
- Sistema de recompensas y castigos.

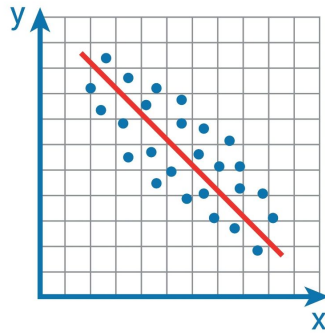


Reinforcement
Learning

Regresión Lineal: Modelando la Tendencia



Positive Correlation



Negative Correlation

- **Objetivo:** Encontrar la relación matemática entre una variable de entrada (X) y una de salida (y).
- **Hipótesis Fuerte:** Asumimos que la relación es lineal. Si X aumenta, y aumenta (o disminuye) de forma constante.
- **Diferencia Clave:**
 - **El Dato:** Es ruidoso y complejo.
 - **El Modelo:** Es una simplificación perfecta (la línea recta).

La Ecuación de la Recta

$$y = w_0 + w_1 x$$

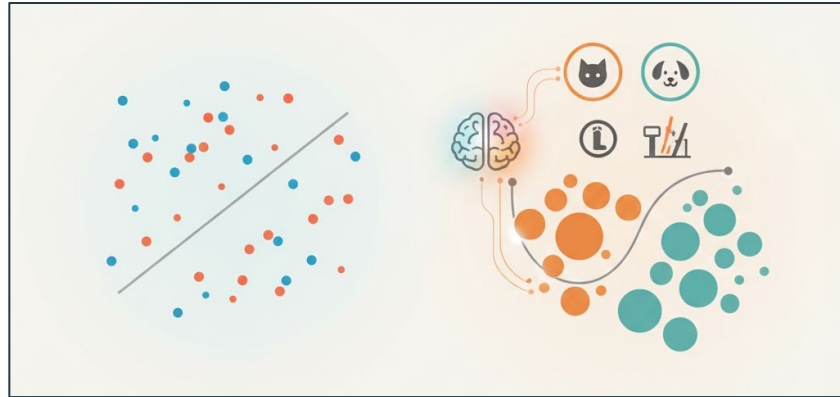
- ❑ y : Predicción.
- ❑ x : Dato de entrada.
- ❑ w_1 (Pendiente/Peso): Cuánto influye la entrada.
- ❑ w_0 (Bias/Sesgo): El punto de partida.

Ejercicios prácticos

1. Resuelve el sobre el Boston Housing Dataset (precios de la vivienda en Boston de los años 70) centrado en la correlación entre Número de Habitaciones y Valor Mediano de la Vivienda.
2. Tras completar el ejercicio anterior, resuelve el "El Predictor de Notas".

El Perceptrón: El Átomo de la IA

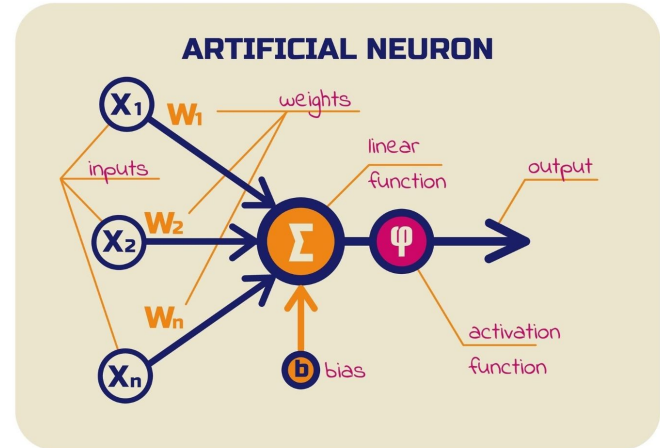
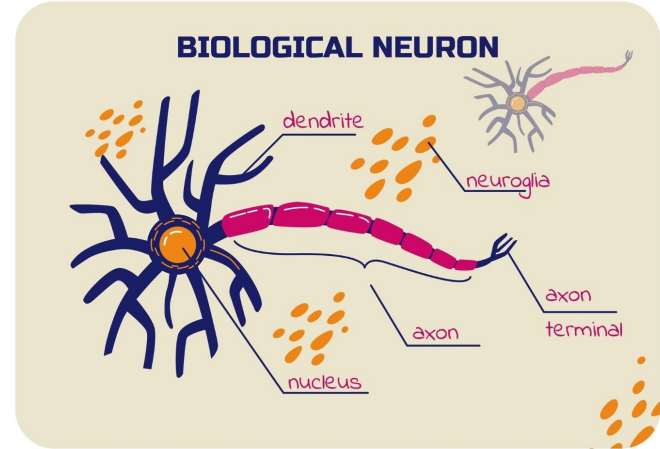
Ya sabemos trazar líneas con la Regresión Lineal. Pero el cerebro humano no solo traza líneas; toma decisiones. '¿Es un perro o un gato?', '¿Freno o acelero?'. Hoy vamos a conocer la unidad mínima de procesamiento que hace esto posible: el Perceptrón.



Imitando al Cerebro

En 1957, Frank Rosenblatt inventó el Perceptrón inspirándose en esto.

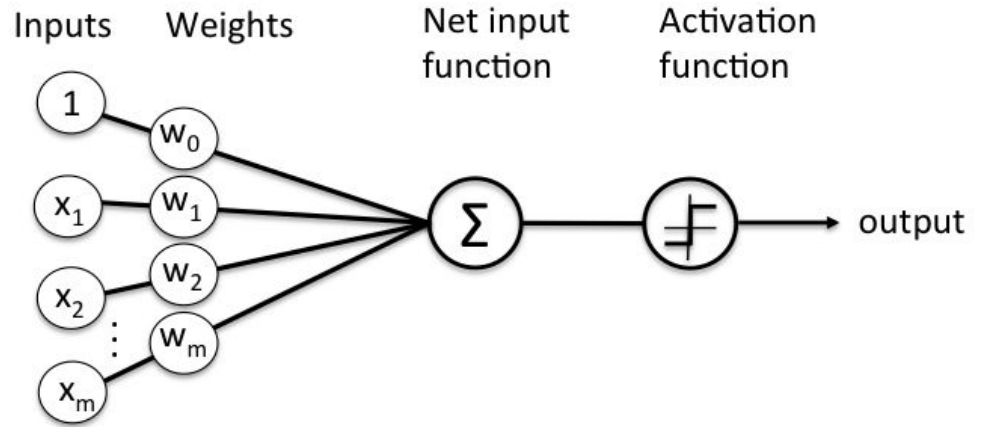
1. **Dendritas:** Reciben señales eléctricas (nuestros Datos de Entrada X).
 2. **Soma (Núcleo):** Acumula la electricidad. Si supera un umbral...
 3. **Axón:** Dispara un chispazo a la siguiente neurona (Salida 1). Si no, se queda callada (Salida 0).
- El Perceptrón es la traducción matemática de este chispazo.



Anatomía del Perceptrón

Matemáticamente, un Perceptrón consta de tres fases:

1. **Ponderación:** Damos importancia a los datos.
2. **Agregación:** Sumamos todo.
3. **Activación:** Decidimos si la suma es suficiente para 'disparar'.



Los Pesos (w): La Memoria del Sistema

Los pesos (w) son el conocimiento acumulado. Son la pendiente de nuestra regresión lineal. Si el peso es alto, esa entrada afecta mucho a la decisión. Si es cercano a cero, la neurona ignora esa entrada. **Aprender no es más que ajustar estos pesos.**

Decisión: *¿Voy a la playa?*

- Entrada 1: ¿Hace sol? (x_1) → Peso MUY ALTO ($w_1 = 5$).
- Entrada 2: ¿Es martes? (x_2) → Peso BAJO ($w_2 = 0.1$).

El Bias (b): El Umbral de Activación

Matemáticamente, el Bias desplaza la función de activación. Sin Bias, nuestra recta de decisión siempre tendría que pasar por el origen (0,0), lo cual nos limitaría muchísimo.

Fórmula: $z = (w \cdot x) + b$

Imaginad que para ir a la playa necesitáis sumar 10 puntos de 'ganar'.

- Los Pesos (w) son el clima (factores externos).
- El Bias (b) es vuestra predisposición personal (factor interno).
 - Si os encanta la playa, vuestro Bias es alto (ya partís con ventaja).
 - Si odiáis la arena, vuestro Bias es negativo (hace falta mucho sol para convencerlos).

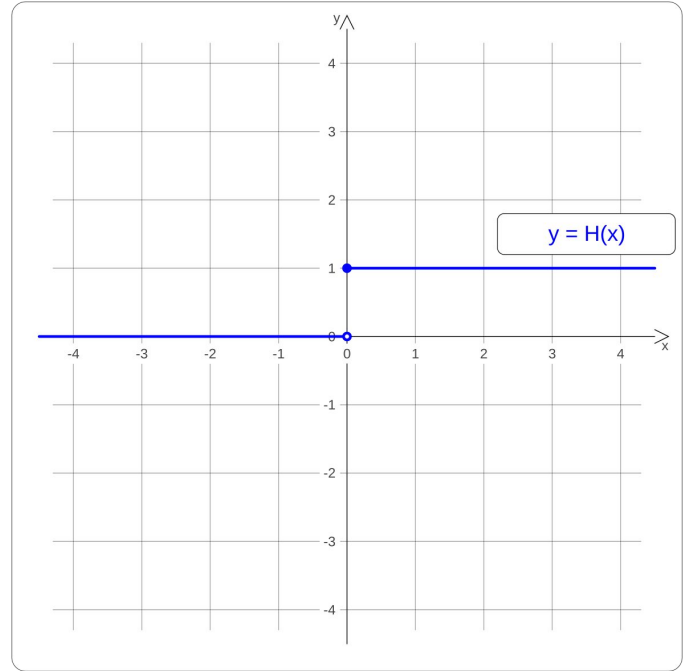
Función de Activación

La regresión lineal nos daba un número (ej: 180.000€). El Perceptrón coge ese número y aplica una regla tajante:

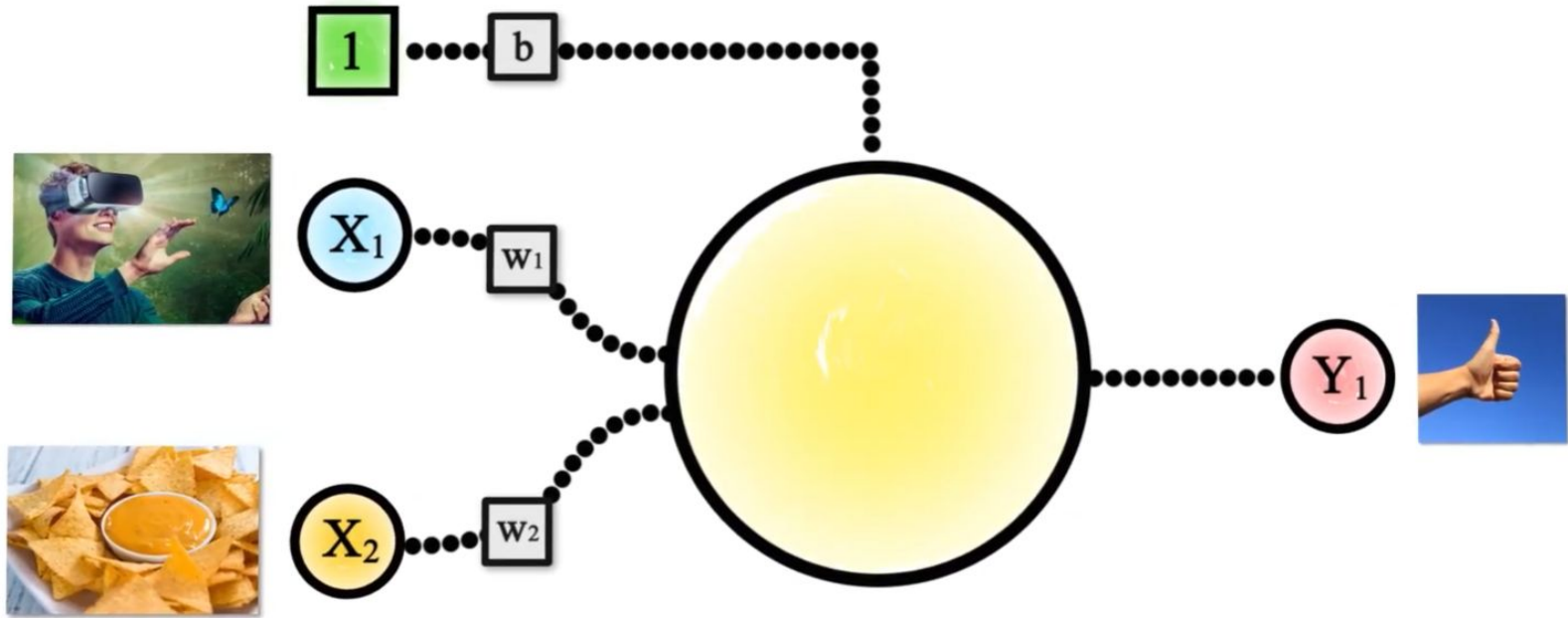
- ¿Es positivo? Devuelvo un 1.
- ¿Es negativo? Devuelvo un 0.

Esta función convierte matemáticas en decisiones lógicas.

$$f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$



VR + NACHOS = PLAN PERFECTO



$$w_1 x_1 + w_2 x_2 + b = y$$

VR + NACHOS = PLAN PERFECTO

VARIABLES BINARIAS

1

0

VR : X_1



Nachos : X_2



VR + NACHOS = PLAN PERFECTO

VARIABLES BINARIAS

EPIC Night : Y_1

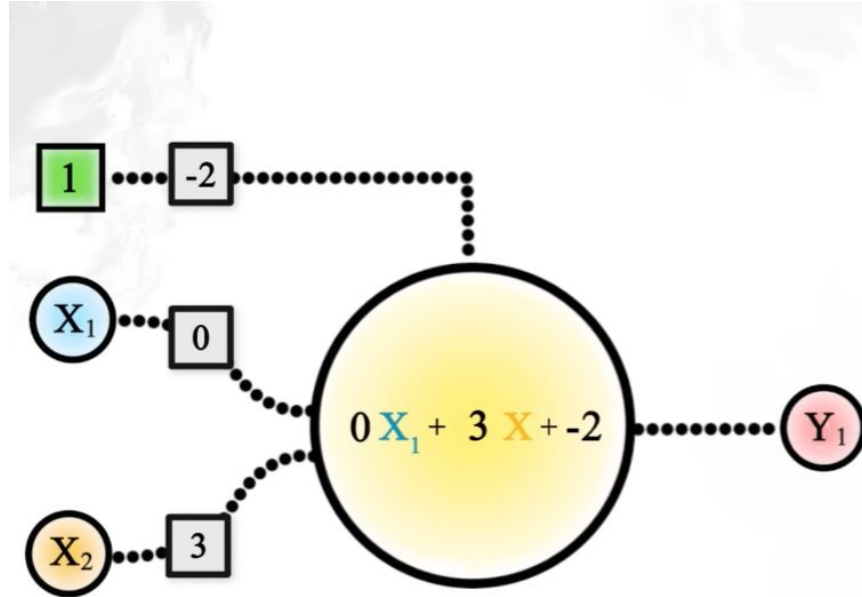
1



0

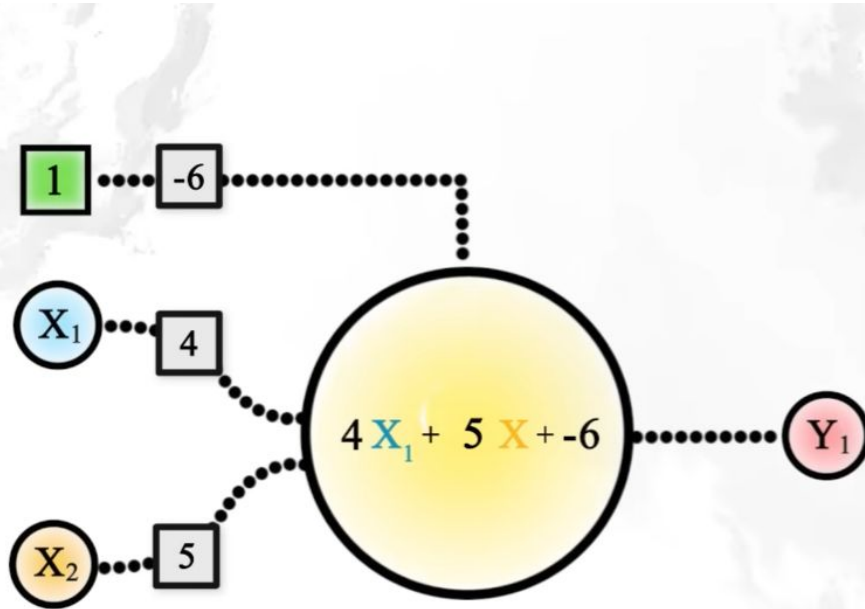


Vamos probando distintos valores...



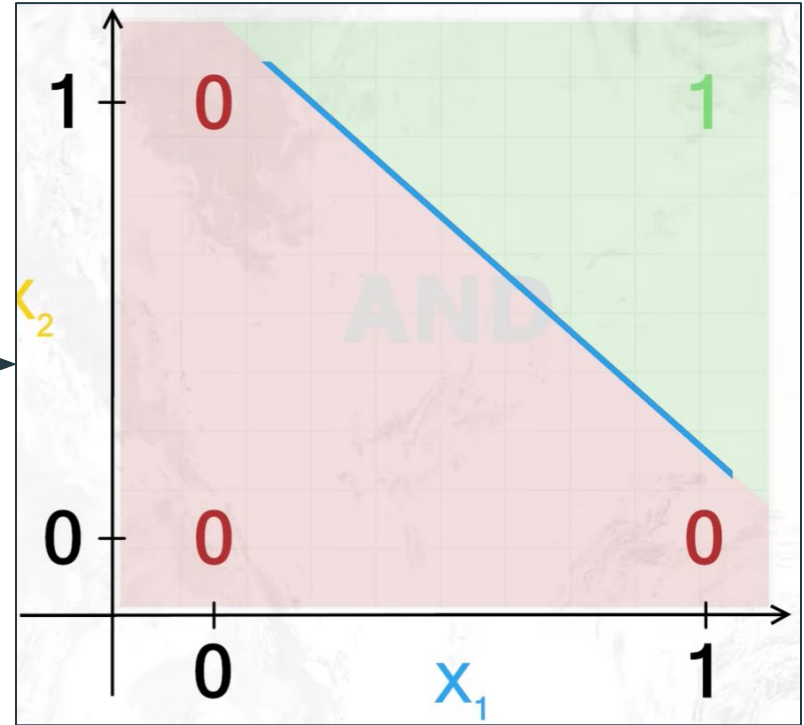
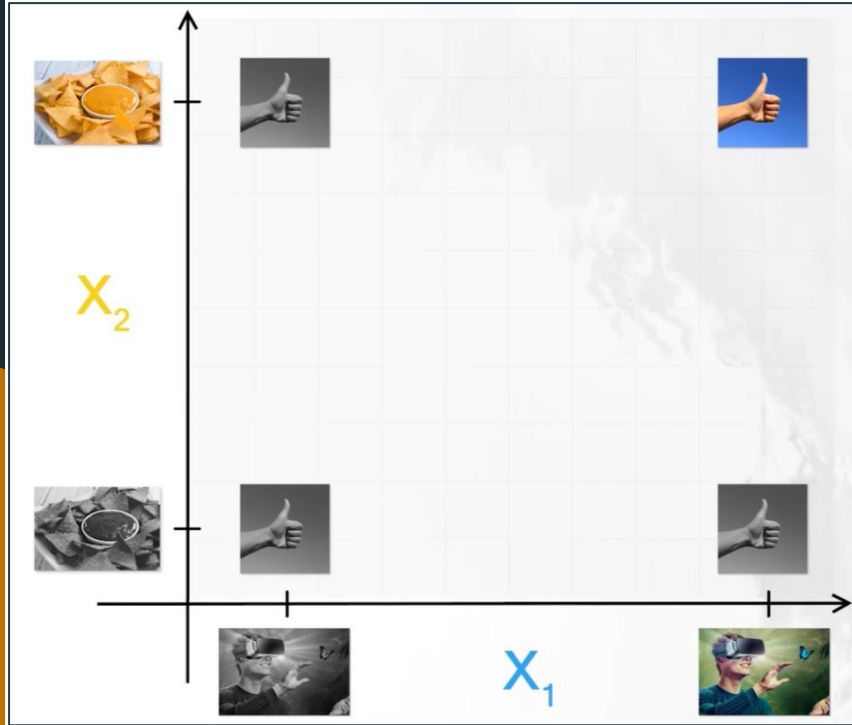
X1	X2	TARGET	Y
			-2
			-2
			1
			1

...hasta que encontremos una combinación ganadora



X1	X2	TARGET	Y
			-6
			-2
			-1
			3

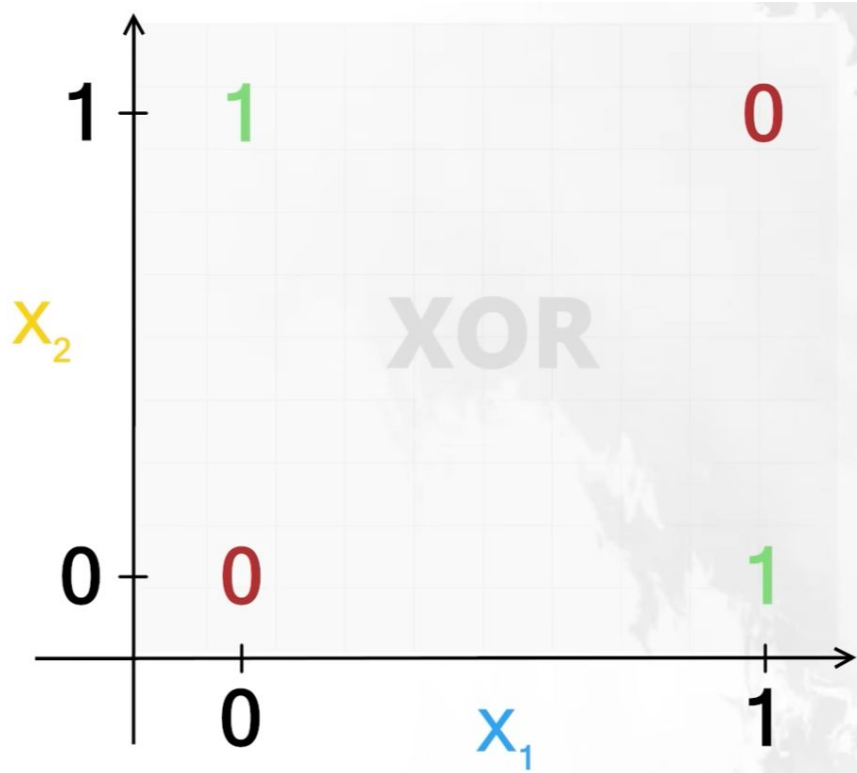
Representación gráfica: puerta lógica AND



Otras posibles gráficas: puerta lógica OR

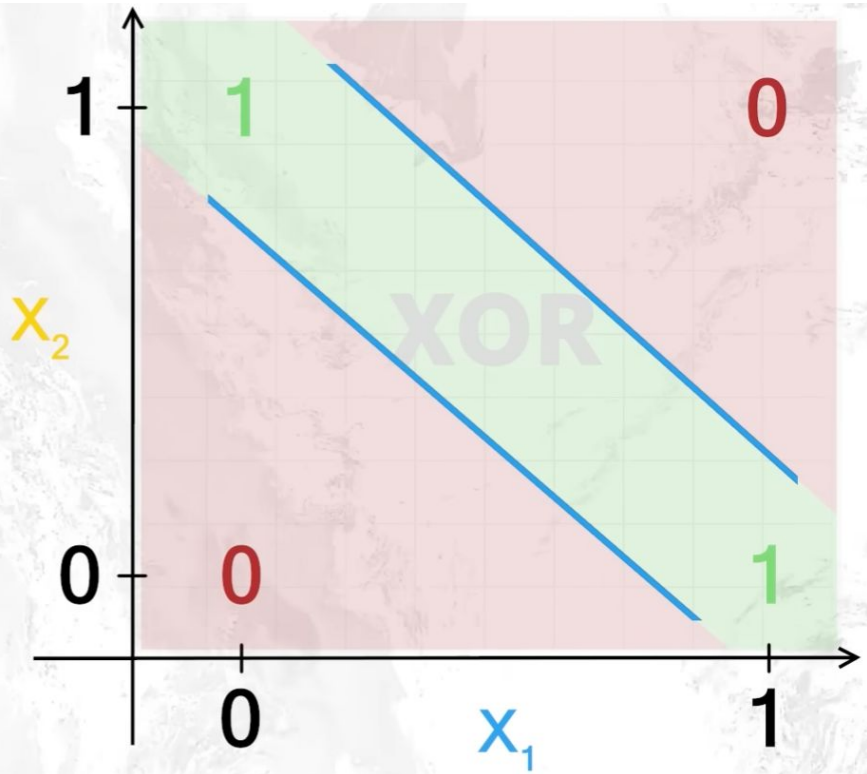
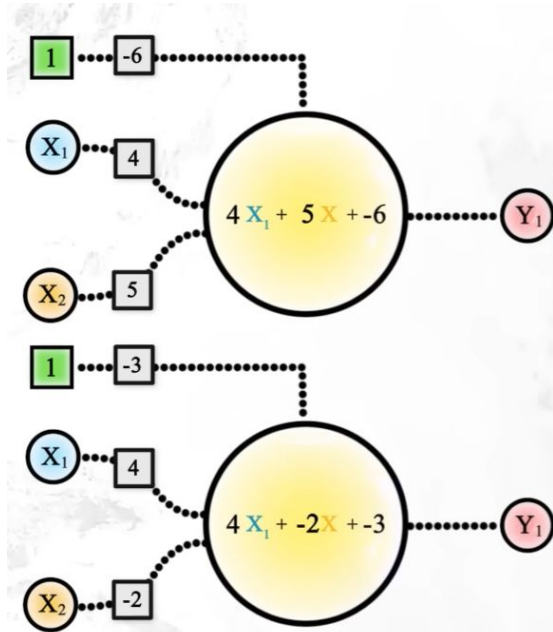


Otras posibles gráficas: puerta lógica XOR



¿Cómo separamos con una línea ambas zonas?

La solución: dos neuronas



Ejercicio práctico: "El Detective de Emails (Anti-Spam)"

Vuestra misión es configurar el Perceptrón que decide si un email es basura o es legítimo. En la vida real, los filtros analizan miles de palabras. Nosotros, para empezar, solo vamos a analizar si aparecen **2 palabras prohibidas**.

Configuración de la Neurona

Entradas (x): (1 si la palabra está, 0 si no está).

- x1: Palabra "**GRATIS**".
- x2: Palabra "**GANADOR**".

Pesos (w): (Nivel de sospecha).

- w1 (Peso de "Gratis") = **0.6** (Bastante sospechoso).
- w2 (Peso de "Ganador") = **0.8** (Muy sospechoso).

Bias (b): -1.0 (Nuestro umbral de tolerancia).

$$z = (x_1 \cdot 0.6) + (x_2 \cdot 0.8) - 1.0$$

- Si $z \geq 0 \rightarrow$ **SPAM (1)**
- Si $z < 0 \rightarrow$ **CORREO NORMAL (0)**

Los Emails (Dataset)

Analiza el texto de los asuntos de los emails y calcula si pasan el filtro:

Email	Asunto del Correo	¿Tiene "Gratis"? (x1)	¿Tiene "Ganador"? (x2)	Cálculo (z)	Veredicto
A	"Hola mamá, ¿qué tal?"	0	0		
B	"¡Eres el GANADOR del sorteo!"	0	1		
C	"Prueba GRATIS nuestro servicio"	1	0		
D	"¡ GANADOR! Recibe tu premio GRATIS "	1	1		

¿Cómo se actualizan los pesos automáticamente?

Si la neurona acierta → No hacemos nada.

Si la neurona falla → Empujamos los pesos en la dirección contraria al error.

La Fórmula (Regla del Perceptrón):

- Calculamos el error: $\text{Error} = (\text{Deseado} - \text{Predicho})$
- Calculamos la corrección: $\text{Update} = \text{TasaAprendizaje} \times \text{Error}$

Actualización:

- $\text{NuevoPeso} = \text{PesoActual} + (\text{Update} \times \text{Entrada})$
- $\text{NuevoBias} = \text{BiasActual} + \text{Update}$

Intuición:

- Si x es positivo y el error es positivo, necesitamos subir el peso.
- La *Tasa de Aprendizaje* (Learning Rate) suele ser pequeña (0.01 o 0.1) para no dar "volantazos".

¿Cómo se actualizan los pesos automáticamente?

Si la neurona acierta → No hacemos nada.

Si la neurona falla → Empujamos los pesos en la dirección contraria al error.

La Fórmula Matemática (Delta Rule):

Para actualizar un peso w_j :

$$w_j := w_j + \Delta w_j$$

Donde el cambio (Δw_j) se calcula así:

$$\Delta w_j = \eta \cdot (y^{(i)} - \hat{y}^{(i)}) \cdot x_j^{(i)}$$

Leyenda de Variables:

- η (**Eta**): Tasa de aprendizaje (**learning rate**). Qué tan rápido corregimos.
- y (**Target**): El valor real (lo que queríamos obtener).
- $\hat{y}$ (**Prediction**): Lo que la neurona predijo.
- x_j : El valor de la entrada asociada a ese peso.

Ejemplo: El Portero de Discoteca

Vamos a imaginar que nuestra neurona es un portero de discoteca muy estricto.

- **Misión:** Solo deja pasar si llevas **Zapatos (x1)** Y **Camisa (x2)**.
- **Problema:** El portero es nuevo y tiene "prejuicios" (pesos) incorrectos: odia a todo el mundo y no deja pasar a nadie.

Estado Inicial (Configuración Errónea)

- **Entrada (x):** [1, 1] (Lleva Zapatos y Camisa). **Debería entrar.**
- **Target (y):** 1 (Entrar).
- **Pesos Iniciales (w):** [-0.5, -0.5] (Pesos negativos = Prejuicio en contra).
- **Bias (b):** 0.
- **Learning Rate (η):** 0.5 (Aprende rápido).

Ejemplo: El Portero de Discoteca

PASO 1: El Intento Fallido (Forward Pass)

Aquí la neurona intenta tomar una decisión con lo que sabe actualmente.

1. Función de Entrada Neta

Calculamos la suma ponderada: $z = (w \cdot x) + b$

$$z = (-0.5 \cdot 1) + (-0.5 \cdot 1) + 0$$

$$z = -0.5 - 0.5 = -1.0$$

2. Función de Activación

Aplicamos la función escalón:

- ¿Es $z \geq 0$?
- -1.0 NO es mayor que 0.
- **Predicción ($\hat{y}$): 0** (No pasas).

Resultado:

- El portero ha dicho "NO" (0).
- La realidad era "SÍ" (1).
- **¡ERROR!** La neurona ha fallado.

Ejemplo: El Portero de Discoteca

PASO 2: El Aprendizaje (Backward Pass)

Como hubo fallo, aplicamos la **Regla de Aprendizaje (Delta Rule)** para corregir los pesos.

1. Cálculo del Error

$$Error = Target - Predicción$$

$$Error = 1 - 0 = +1$$

(El error positivo significa: "Te has quedado corto, deberías haber sumado más").

Ejemplo: El Portero de Discoteca

2. Cálculo de la Corrección (Δw)

Fórmula: $\Delta w = \eta \cdot \text{Error} \cdot \text{Input}$

- Para el Peso 1 (Zapatos):
 $\Delta w_1 = 0.5 \cdot 1 \cdot 1 = +0.5$ (Como llevabas zapatos y te dije que no, debo valorar más los zapatos).
- Para el Peso 2 (Camisa):
 $\Delta w_2 = 0.5 \cdot 1 \cdot 1 = +0.5$
- Para el Bias:
 $\Delta b = 0.5 \cdot 1 = +0.5$

3. Actualización de Pesos

$$w_{nuevo} = w_{viejo} + \Delta w$$

- $w_1 (\text{nuevo}) = -0.5 + 0.5 = 0.0$
- $w_2 (\text{nuevo}) = -0.5 + 0.5 = 0.0$
- $b (\text{nuevo}) = 0 + 0.5 = 0.5$

Ejemplo: El Portero de Discoteca

PASO 3: La Verificación (¿Funciona ahora?)

Vamos a probar otra vez con la **misma entrada** [1, 1] pero con los **nuevos pesos**.

1. Nueva Entrada Neta

$$z = (0.0 \cdot 1) + (0.0 \cdot 1) + 0.5$$

$$z = 0 + 0 + 0.5 = 0.5$$

2. Nueva Activación (predict)

- ¿Es $z \geq 0$?
- 0.5 Sí es mayor que 0.
- **Predicción: 1** (Adentro).

CONCLUSIÓN:

La neurona ha aprendido. En un solo paso, ha pasado de rechazar injustamente al cliente a dejarlo pasar. Ha ajustado sus "prejuicios" (pesos) basándose en el error cometido.

Práctica 1

Es el momento de llevar la teoría a la práctica. Vamos a programar el Perceptrón de Rosenblatt.

Entra en Moodle y realiza la actividad *Práctica 1: Programando el Perceptrón de Rosenblatt*

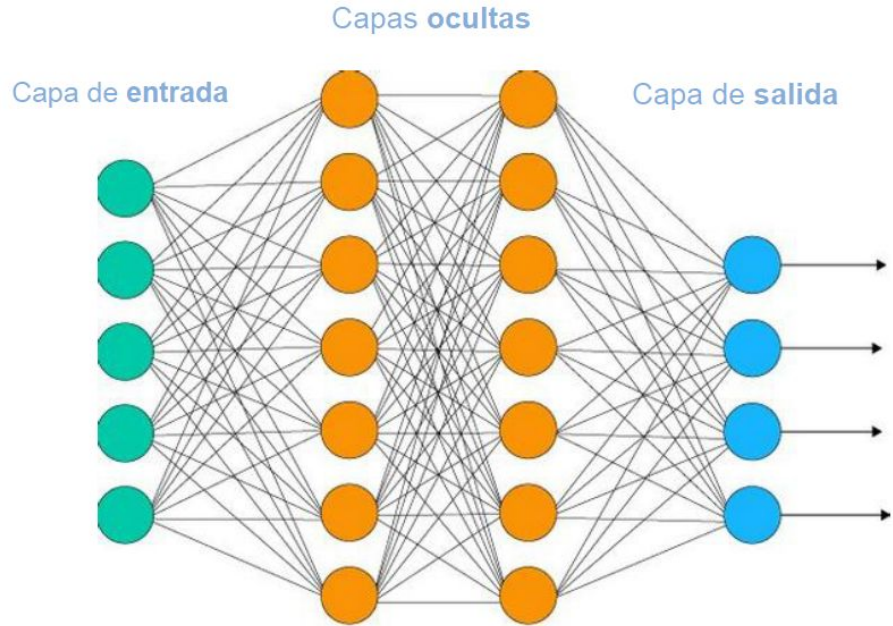
Red Neuronal

Problemas sencillos → 1 Neurona

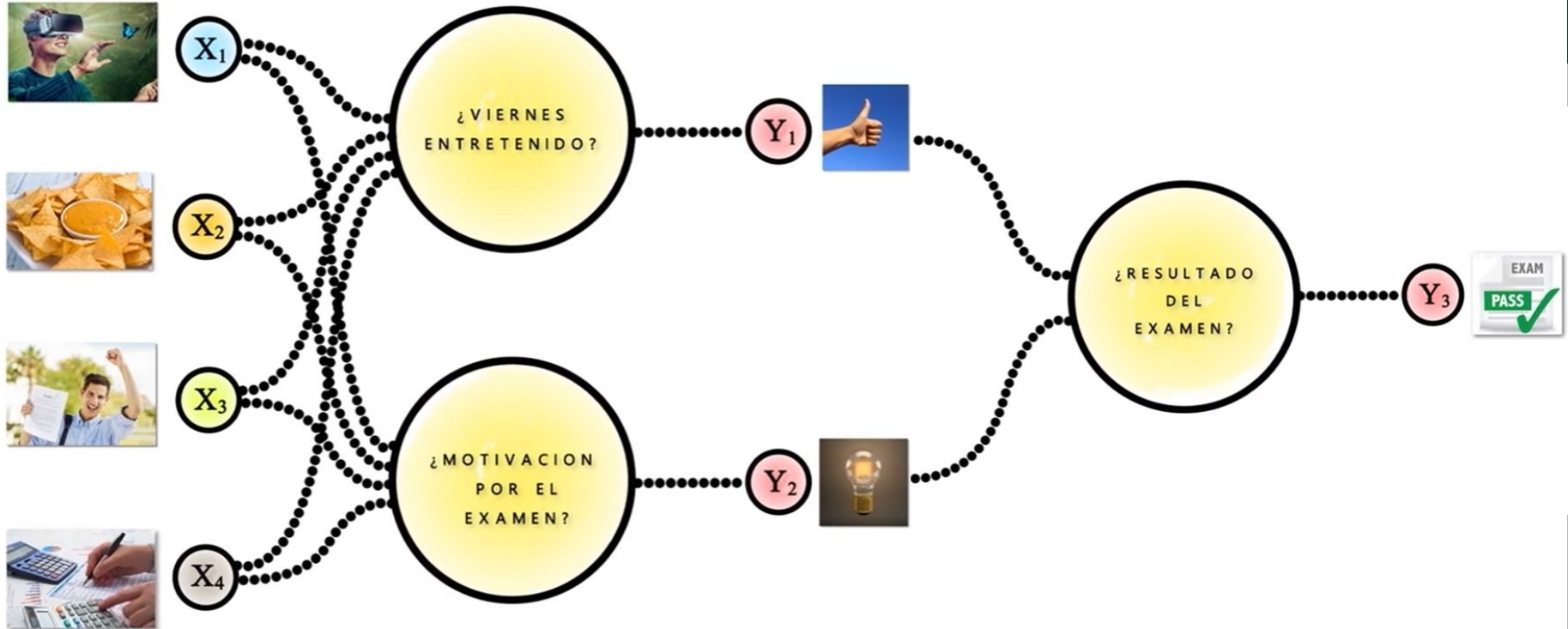
Problemas complejos → Red Neuronal

Propiedades importantes:

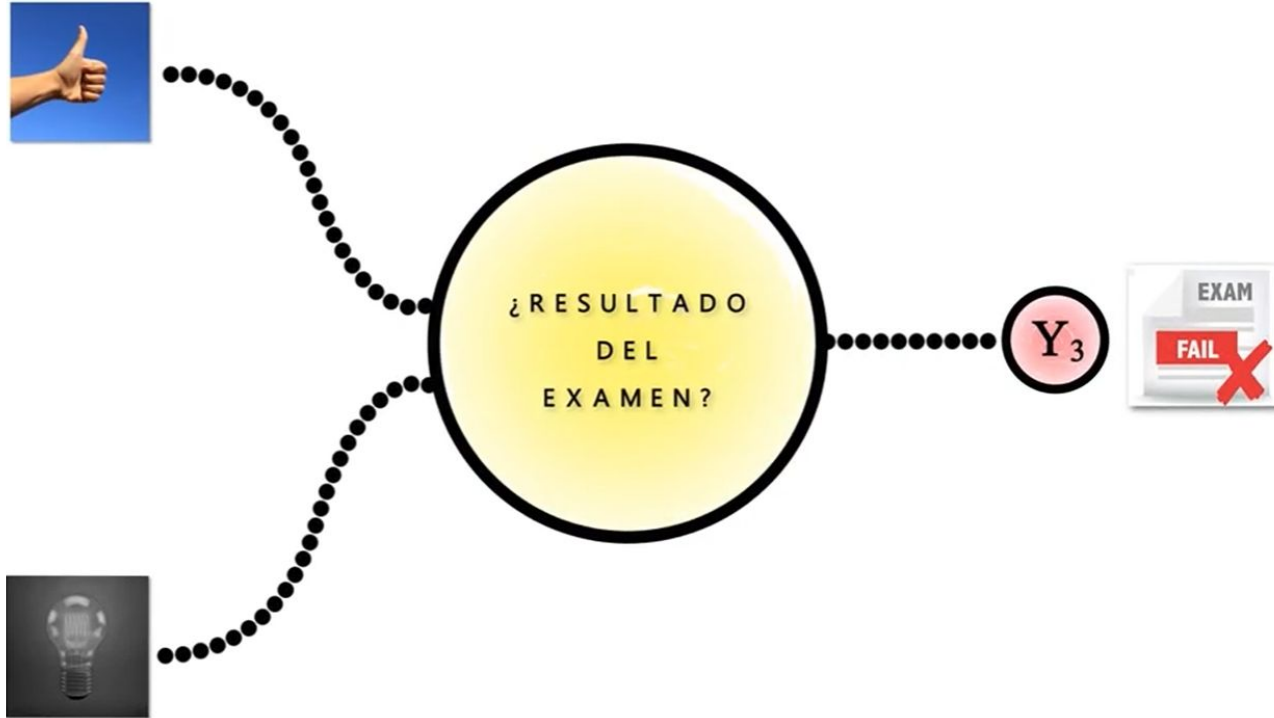
- Organización **por capas**.
- Puede haber **distinto número de neuronas por capa**.
- Cantidad de **capas variables** en función del problema
- **Todas las neuronas** de una capa están **conectadas** con todas las neuronas de su capa anterior y posterior



Ampliamos el ejemplo anterior

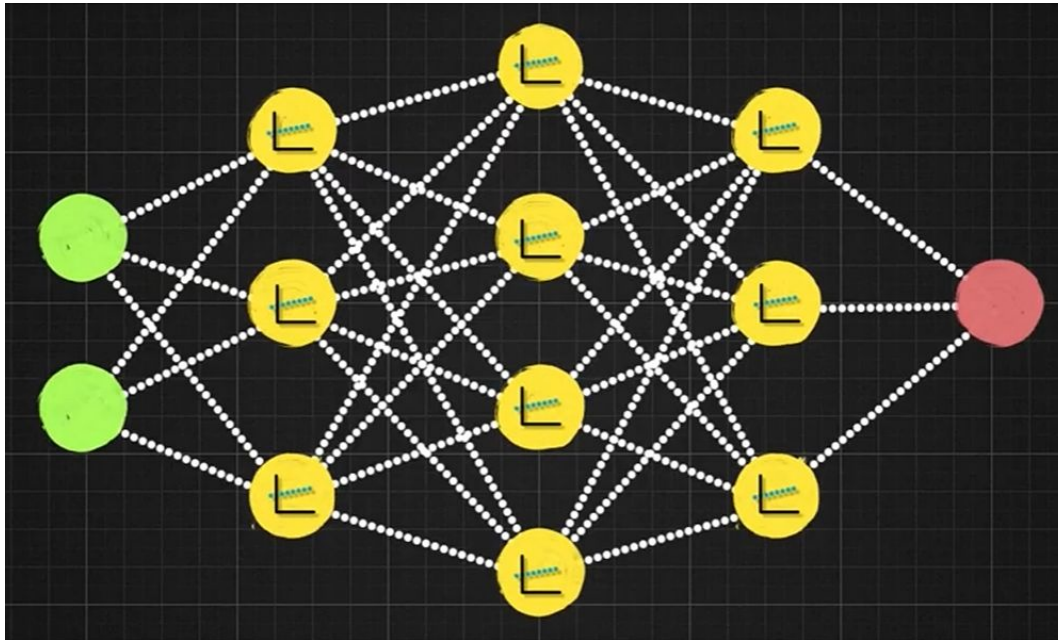


Ampliamos el ejemplo anterior



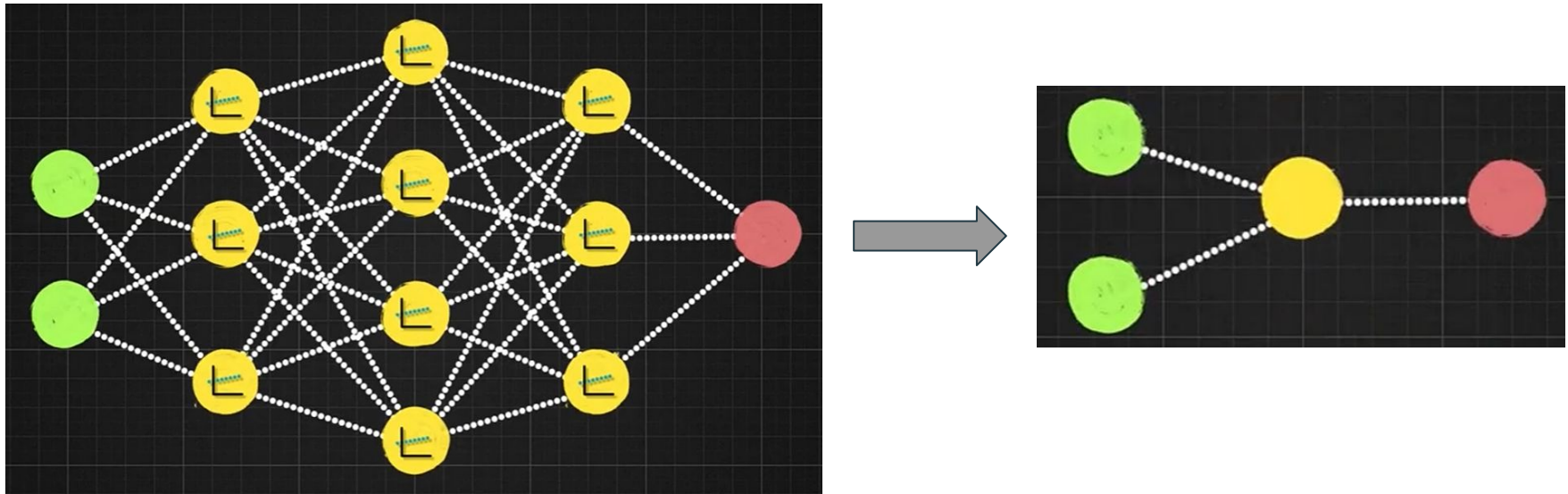
Inconveniente

Hasta ahora, hemos visto que lo que hace cada neurona es resolver un problema de regresión lineal.



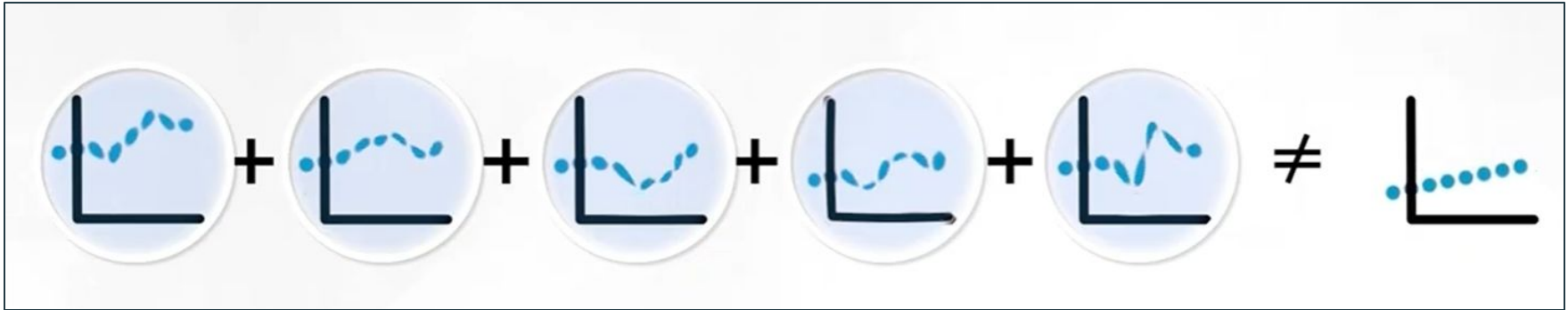
Inconveniente

Hasta ahora, hemos visto que lo que hace cada neurona es resolver un problema de regresión lineal. Esto provoca que toda la estructura que queríamos conseguir colapse hasta ser equivalente a tener una única neurona.

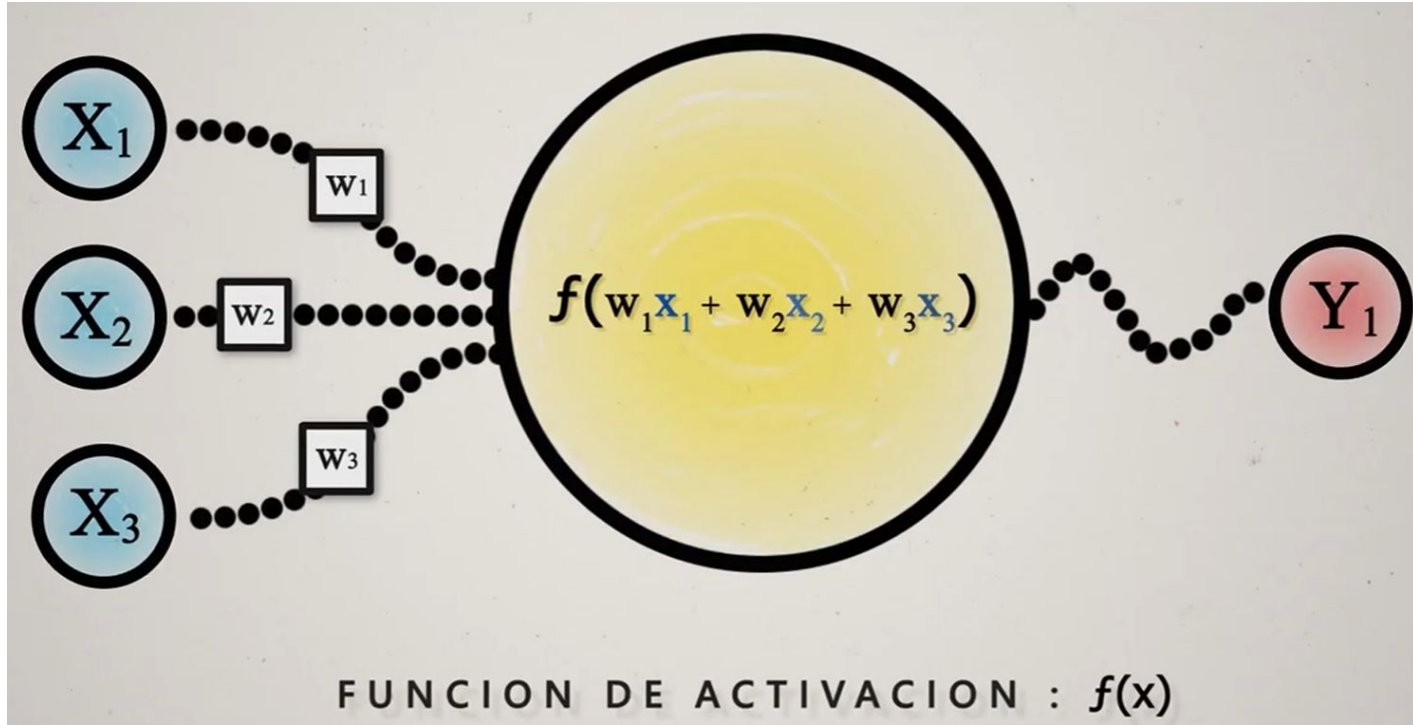


La solución

Para que no colapse, necesitamos que la suma de todas las neuronas de como resultado algo diferente a una línea recta. Necesitamos que todas las líneas sufra alguna manipulación no lineal que las distorsione.



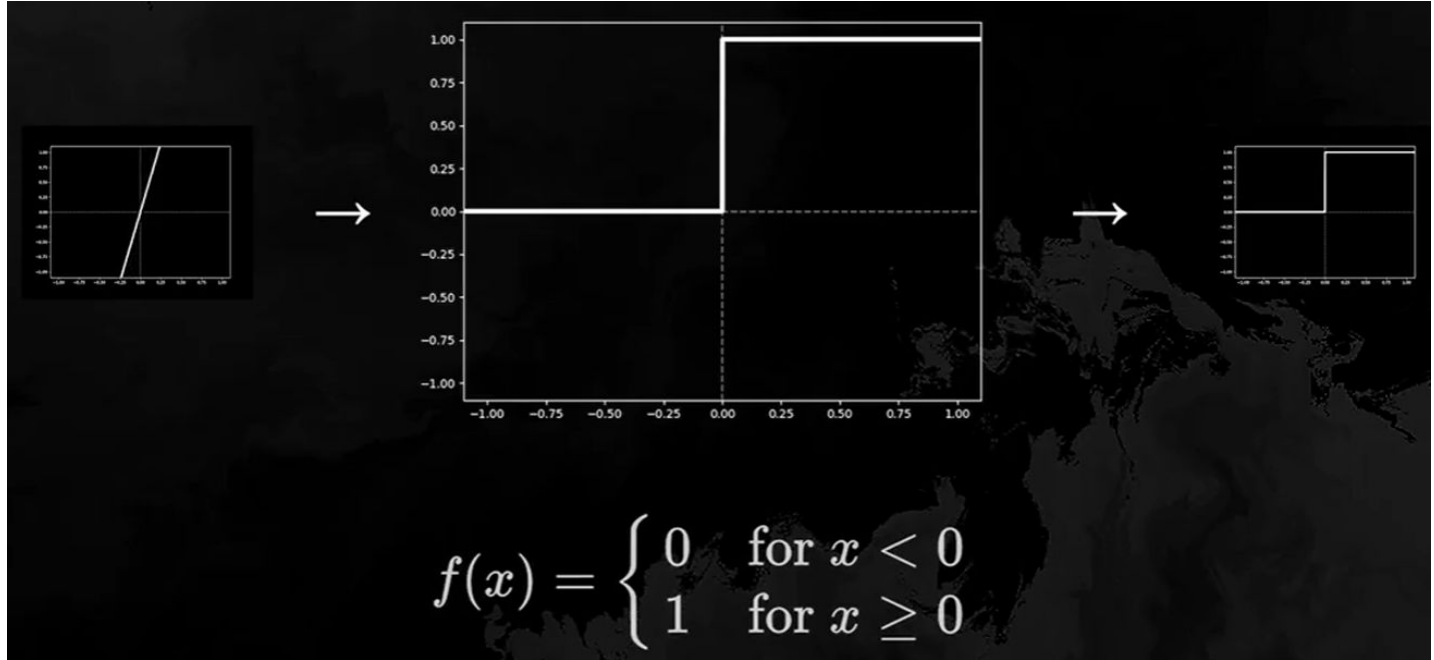
Funciones de activación



Función escalonada

El cambio de valor se produce instantáneamente y no de forma gradual.

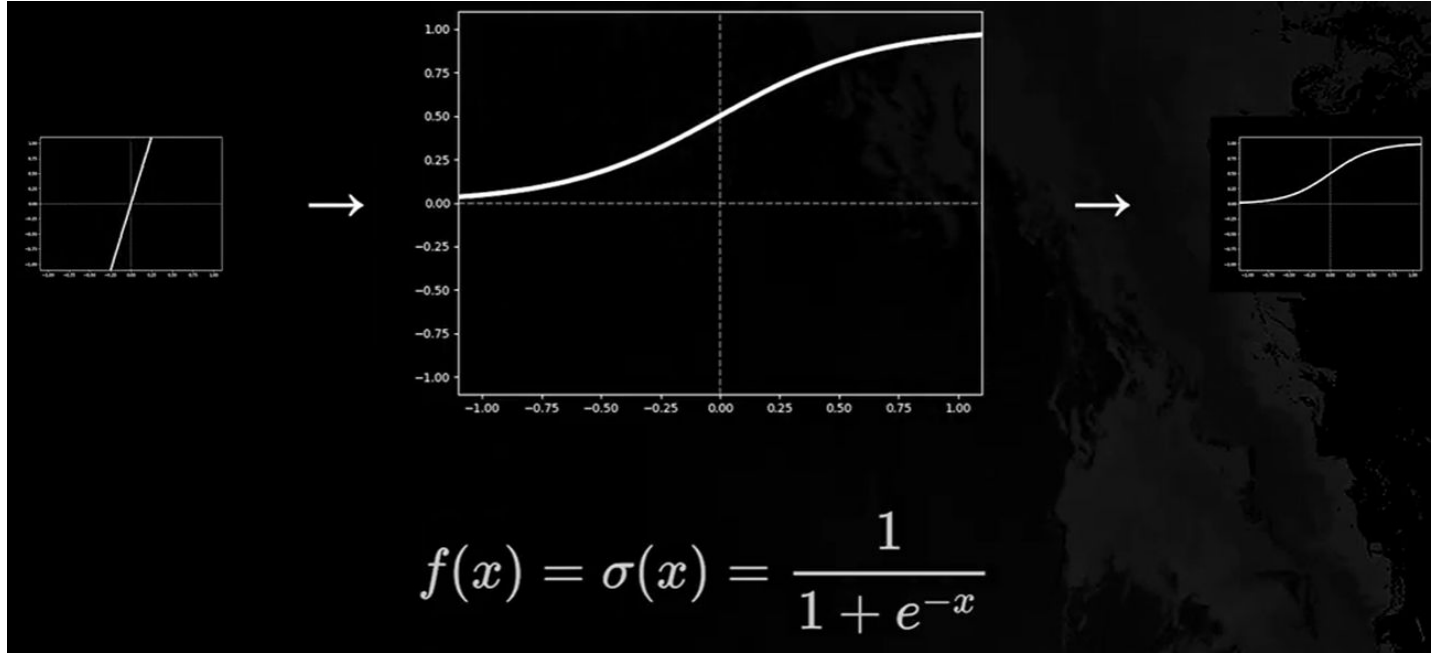
Veremos más adelante que no favorece el aprendizaje.



Función sigmoide

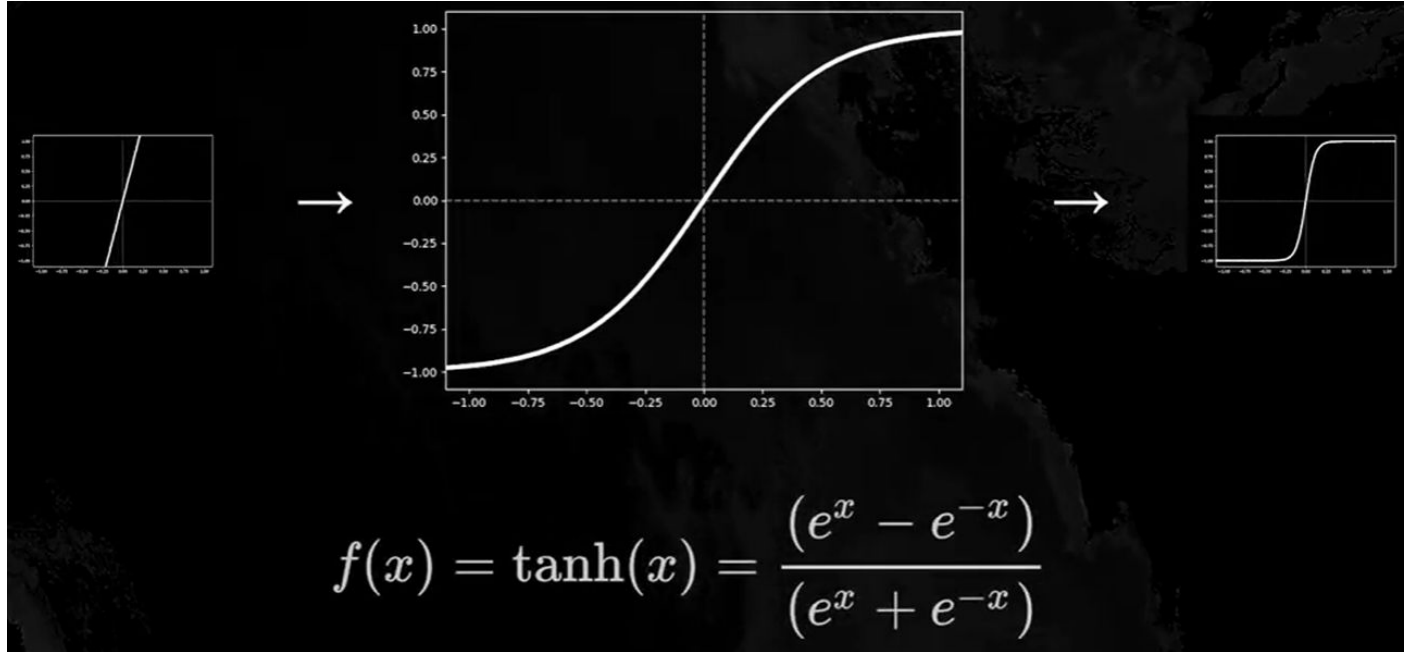
Hace que los valores muy grandes se saturen en 1 y los muy pequeños en 0.

Sirve para representar probabilidades, ya que siempre vienen en el rango de 0 a 1.



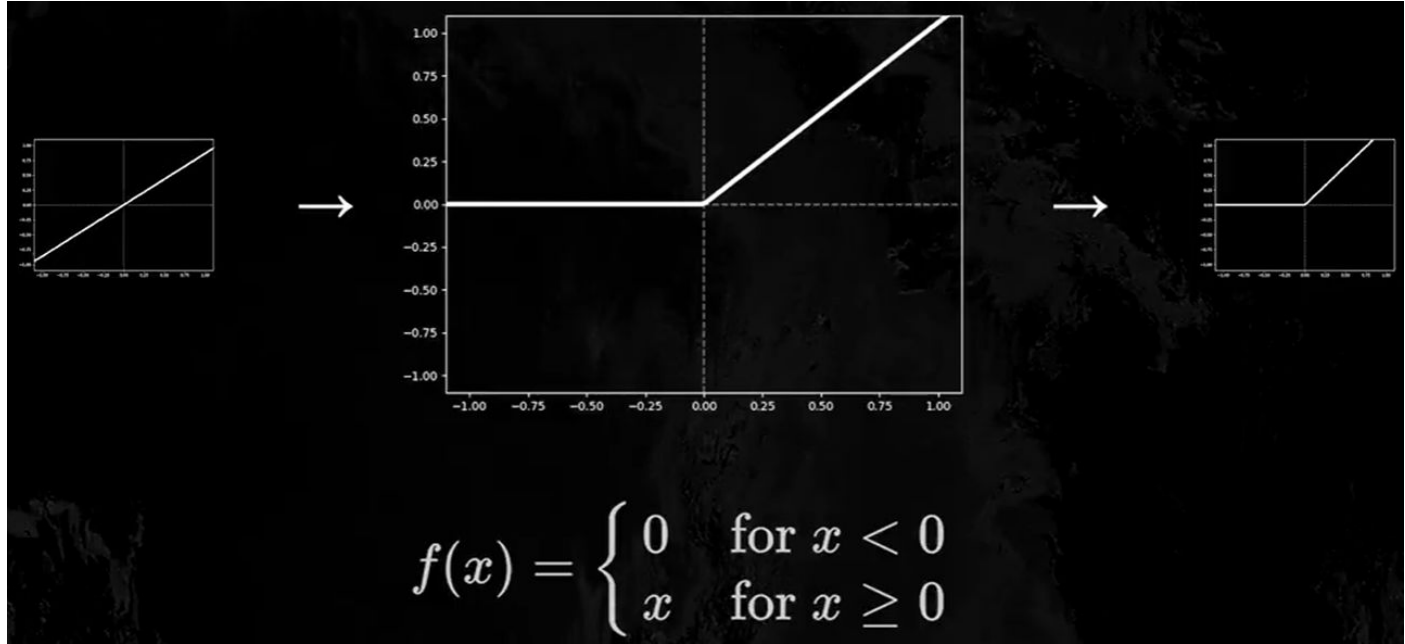
Función tangente hiperbólica

Parecida a la sigmoide pero el rango varía de -1 a 1.

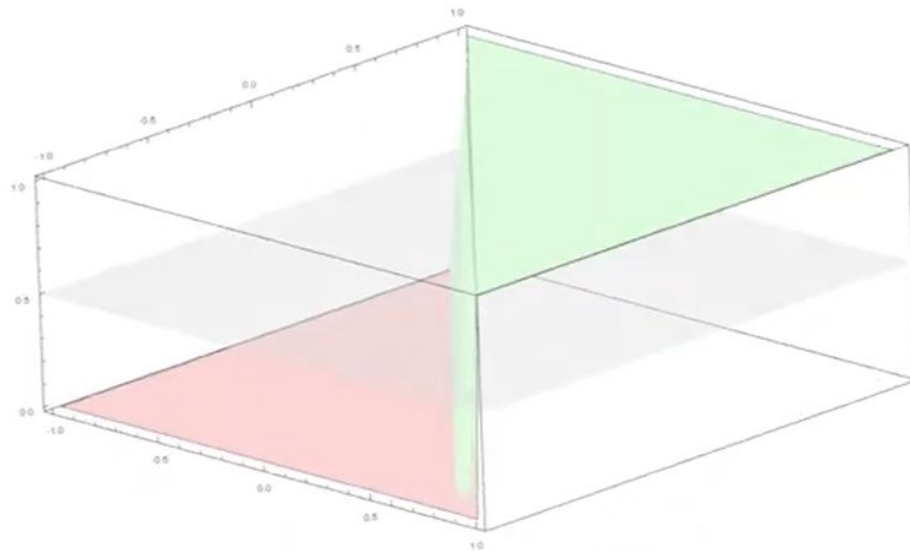
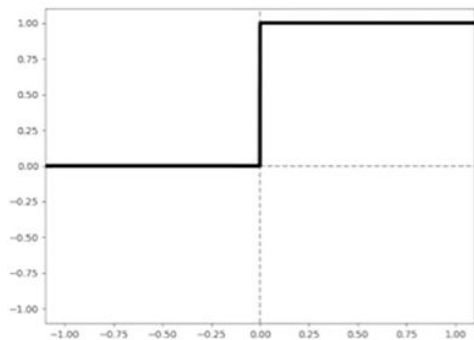


Unidad rectificada lineal

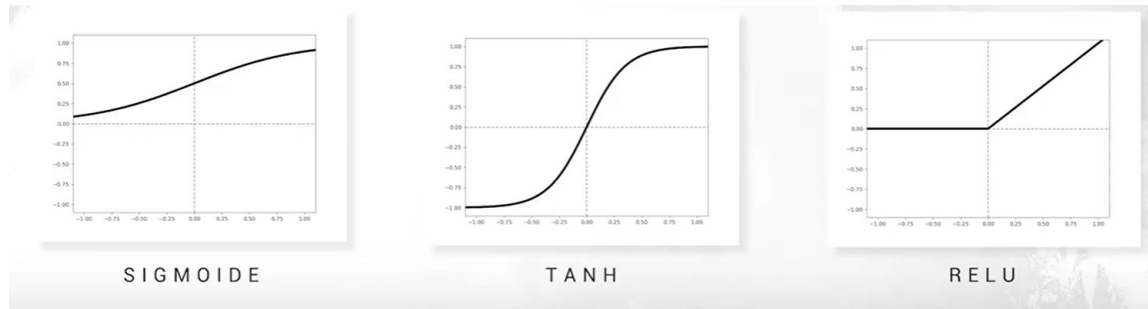
Se comporta como una función lineal cuando el valor de entrada es positivo y constante a 0 cuando es negativo.



Interpretación geométrica

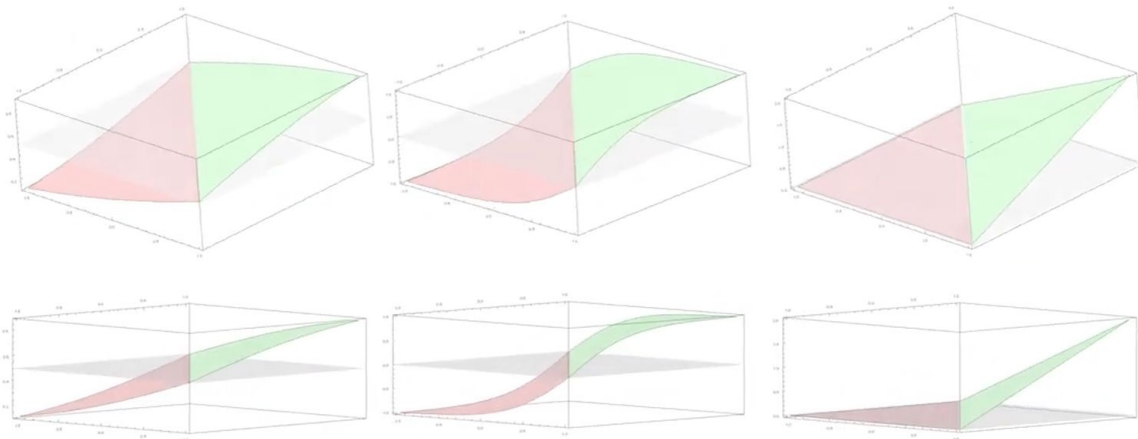


Interpretación geométrica

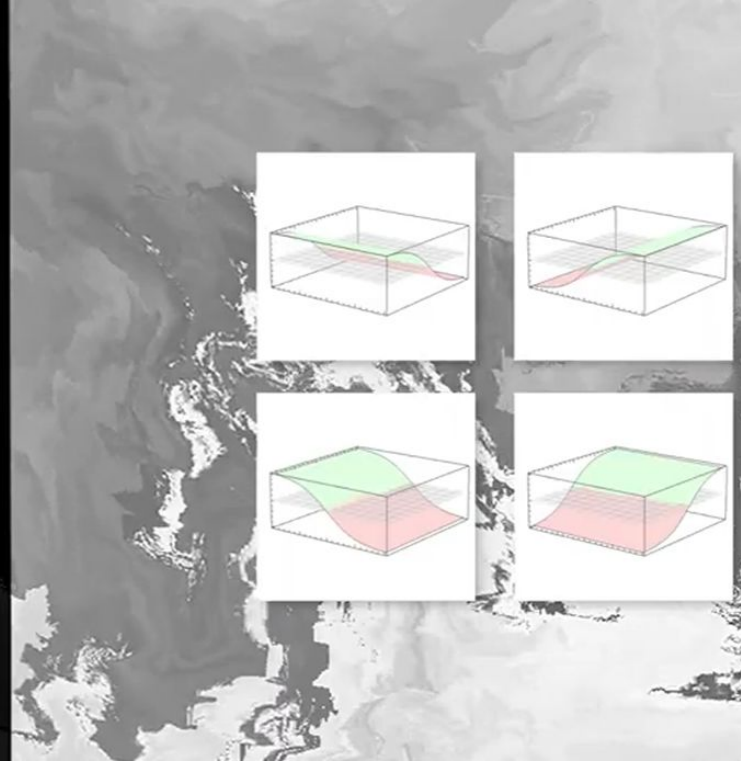
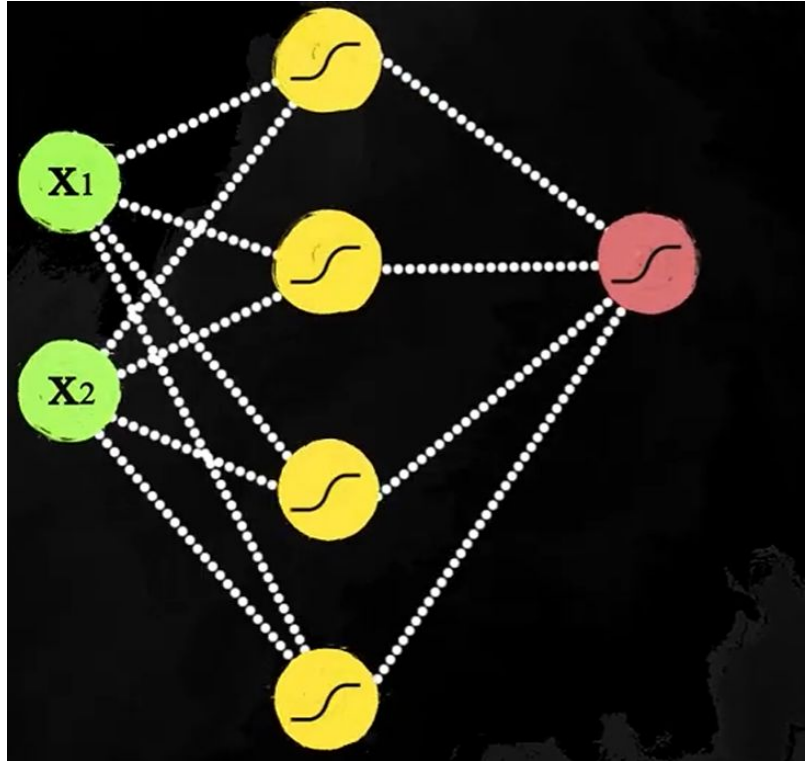


2D

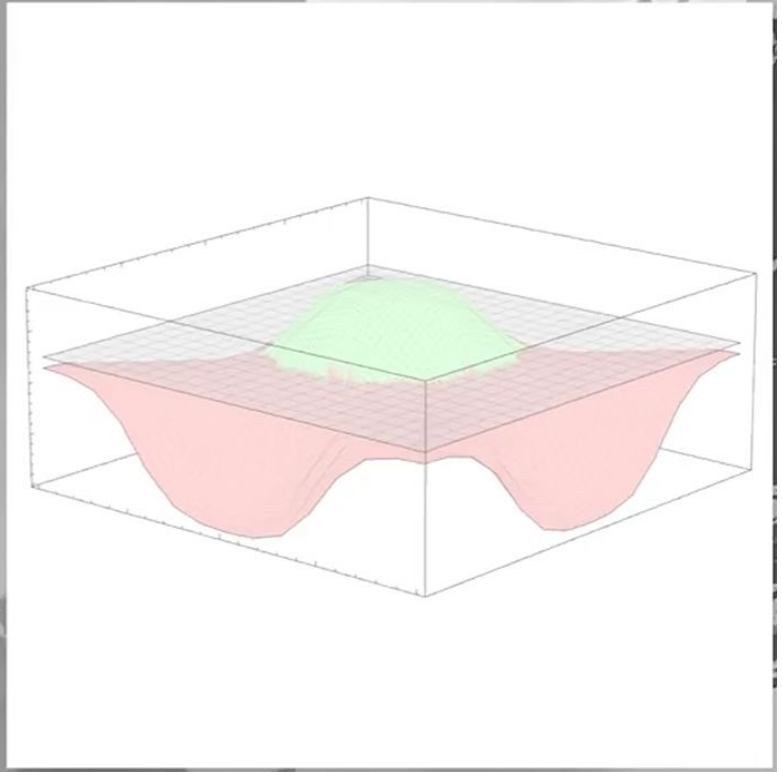
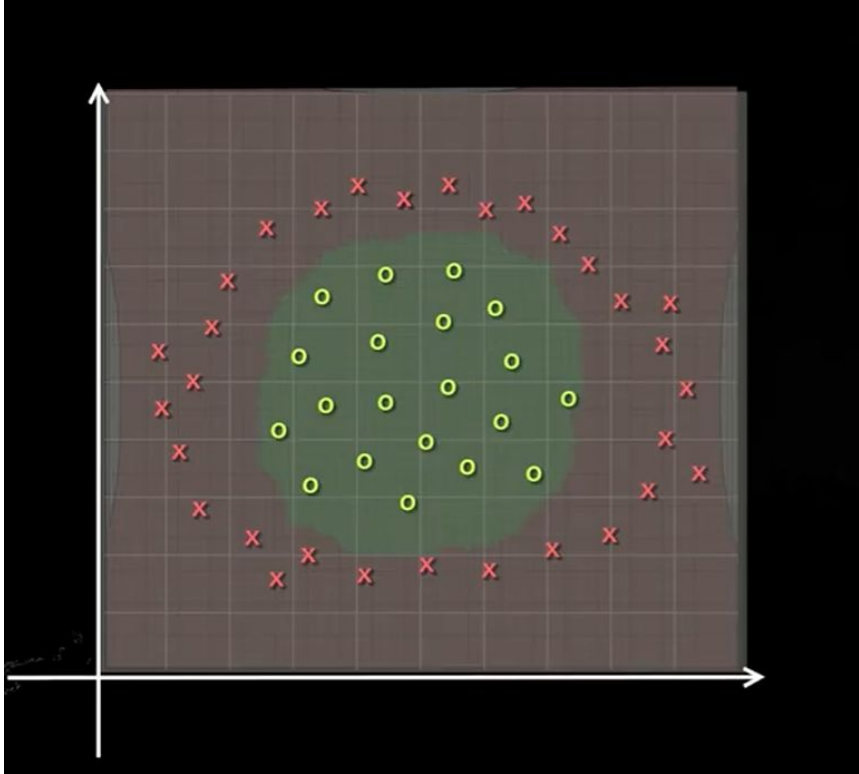
3D



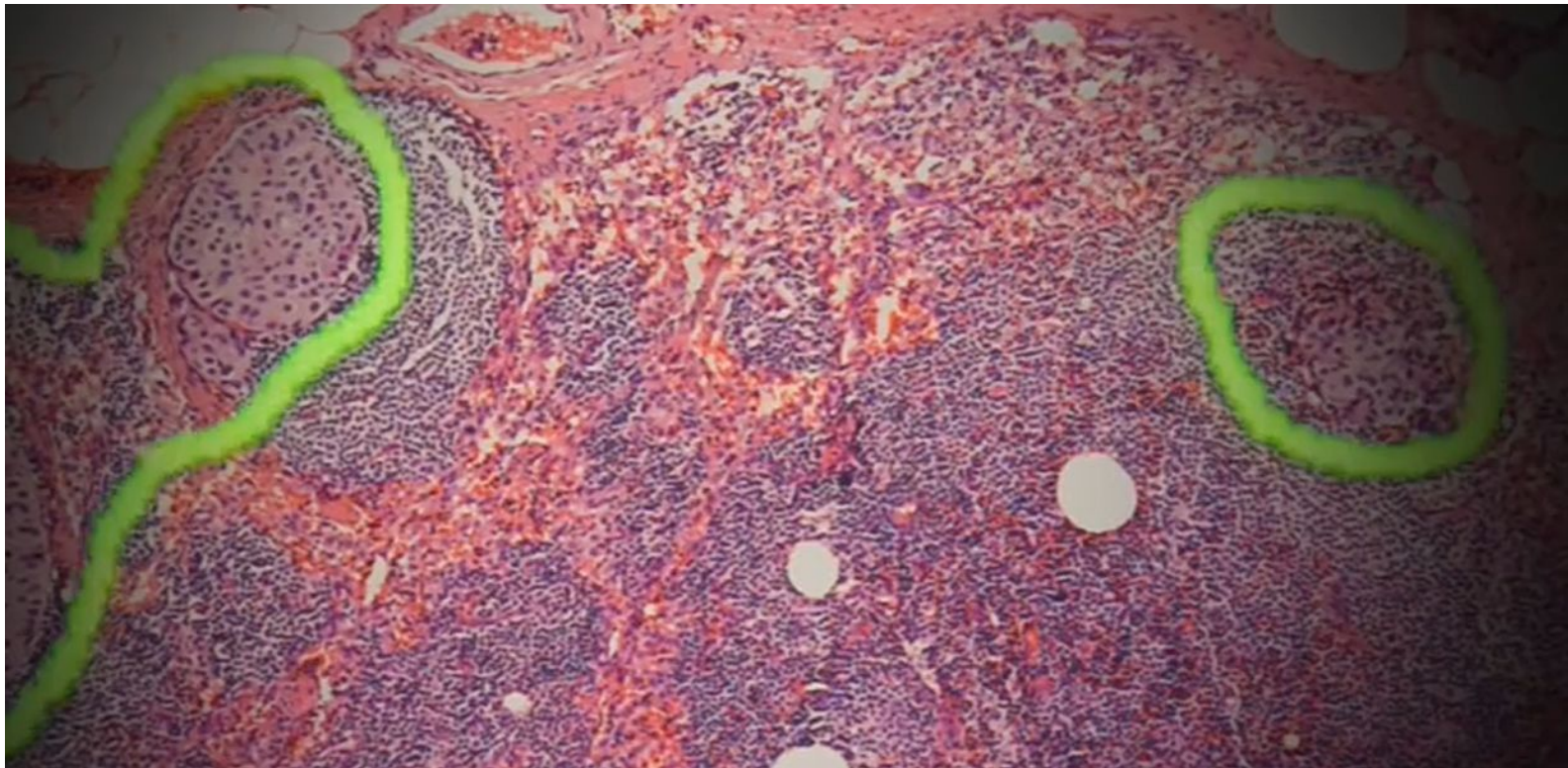
Aplicándolo a una red neuronal



Aplicándolo a una red neuronal



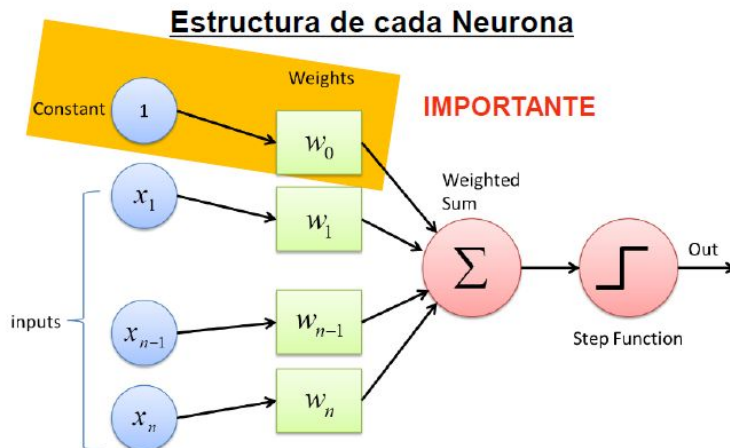
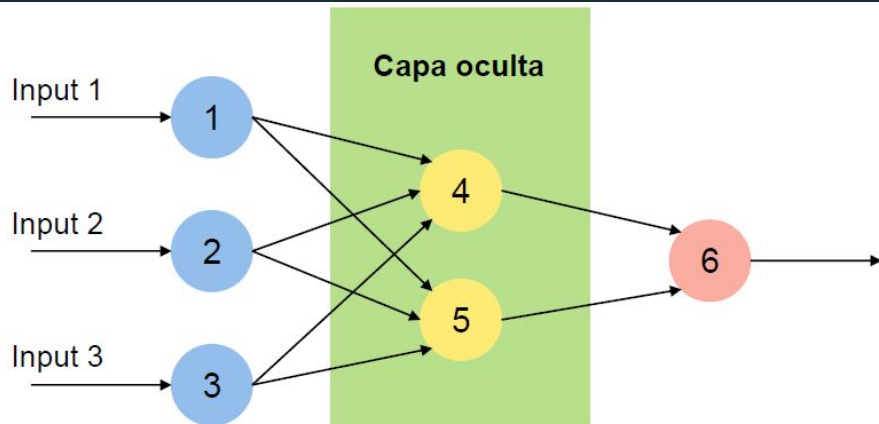
Aplicándolo a una red neuronal



Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los **enviaremos directamente a todas las neuronas** de la primera capa oculta.

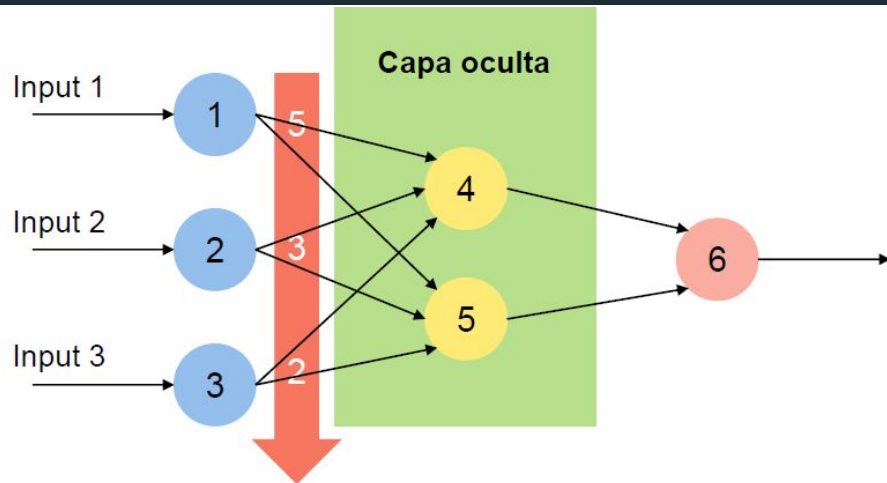
- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos de todas las neuronas de la capa**
- El **resultado** será **una matriz** con los **resultados de cada neurona**
- Aplicaremos la **misma función de activación** a **todas las neuronas de una capa**



Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los enviaremos directamente a todas las neuronas de la primera capa oculta.

- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos** de todas las neuronas de la capa
- El **resultado** será una **matriz** con los **resultados de cada neurona**
- Aplicaremos la **misma función de activación** a todas las neuronas de una capa



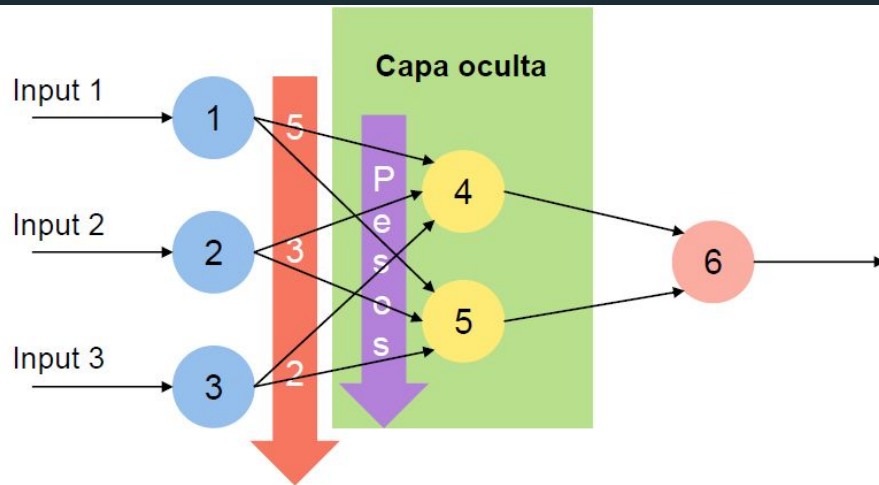
Los **Outputs** de la capa anterior serán **Inputs** de nuestra capa

Constante	Input 1	Input 2	Input 3
1	5	3	2

Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos **los enviaremos directamente a todas las neuronas** de la primera capa oculta.

- Los **inputs** de la capa (outputs de la capa anterior) serán una **matriz**
- Otra **matriz** serán los **pesos** de todas las neuronas de la capa
- El **resultado** será una **matriz** con los **resultados** de cada neurona
- Aplicaremos la **misma función de activación** a todas las neuronas de una capa



Los **Outputs** de la capa anterior serán **Inputs** de nuestra capa

Constante	Input 1	Input 2	Input 3
1	5	3	2

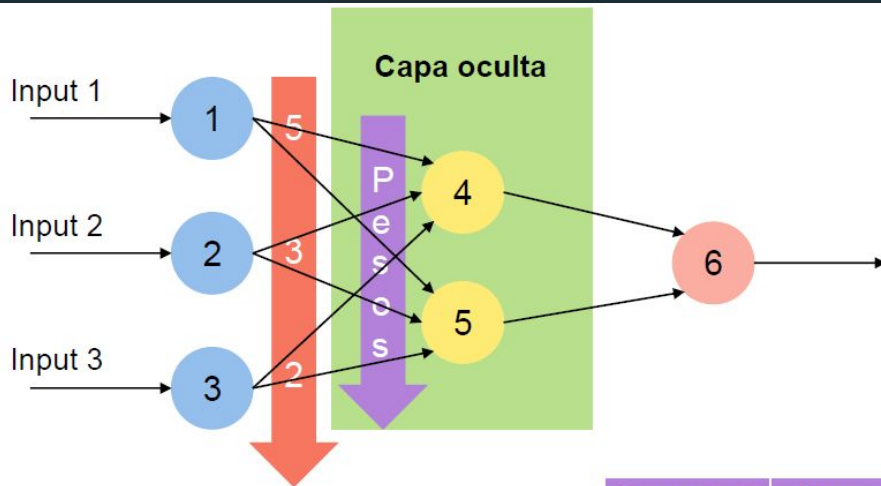
Los **Pesos** de la **capa** serán una **matriz de dos dimensiones**

	Neurona 4	Neurona 5
Bias	-2	3
Peso 1	1	0
Peso 2	-2	-1
Peso 3	0	3

Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los **enviaremos directamente a todas las neuronas** de la **primera capa oculta**.

- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos** de todas las neuronas de la capa
- **El resultado será una matriz con los resultados de cada neurona**
- Aplicaremos la **misma función de activación** a todas las neuronas de una capa



1 5 3 2



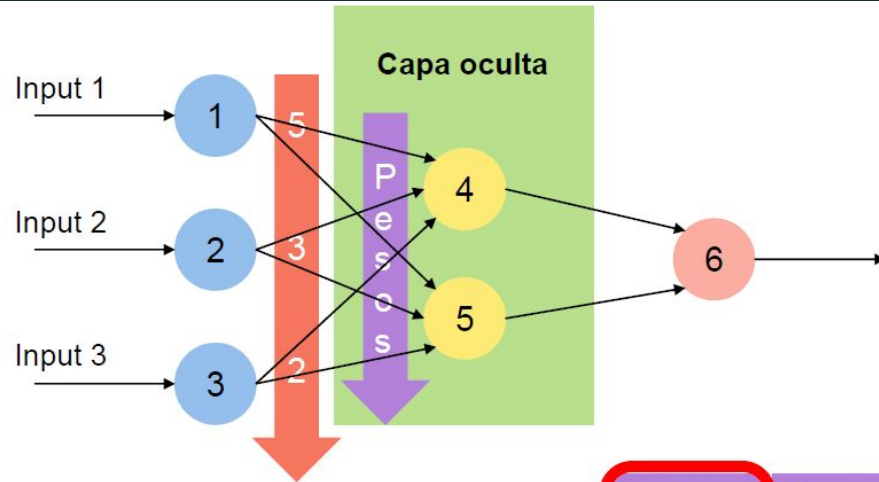
	Neurona 4	Neurona 5
Bias	-2	3
Peso 1	1	0
Peso 2	-2	-1
Peso 3	0	3

Suma ponderada
Calculada mediante
una multiplicación
matricial

Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los enviaremos **directamente a todas las neuronas** de la primera capa oculta.

- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos** de todas las neuronas de la capa
- El **resultado** será **una matriz** con los **resultados de cada neurona**
- Aplicaremos la **misma función de activación** a todas las neuronas de una capa



1	5	3	2
---	---	---	---



	Neurona 4	Neurona 5
Bias	-2	3
Peso 1	1	0
Peso 2	-2	-1
Peso 3	0	3

Suma ponderada
Calculada mediante
una multiplicación
matricial

Cálculo

$$1 \times -2 + 5 \times 1 + 3 \times -2 + 2 \times 0 = -3$$

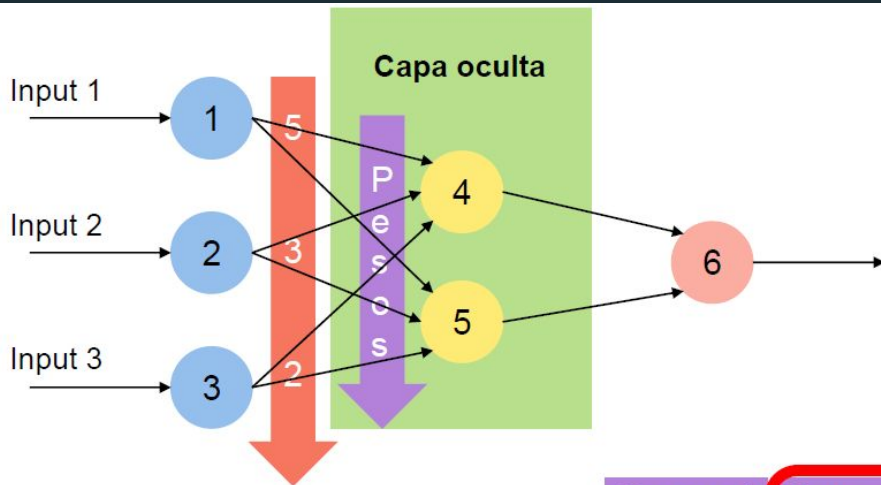
Resultado

-3

Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los **enviaremos directamente a todas las neuronas** de la **primera capa oculta**.

- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos** de todas las neuronas de la capa
- **El resultado será una matriz con los resultados de cada neurona**
- Aplicaremos la **misma función de activación** a todas las neuronas de una capa



1	5	3	2
---	---	---	---



	Neurona 4	Neurona 5
Bias	-2	3
Peso 1	1	0
Peso 2	-2	-1
Peso 3	0	3

Suma ponderada
Calculada mediante
una multiplicación
matricial

Cálculo

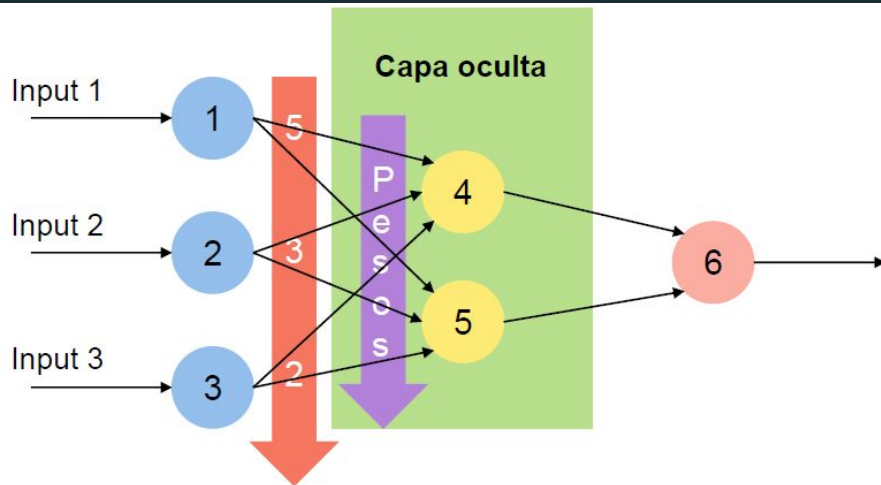
$$1 \times 3 + 5 \times 0 + 3 \times -1 + 2 \times 3 = 6$$

Resultado	
-3	6

Programando una Capa Neuronal

En nuestro caso supondremos que tenemos una **primera capa neuronal** en nuestra red que no haga **ningún tipo de transformación** sobre los datos de entrada. De esta forma, **simplificamos su programación** y simplemente los **inputs** que recibamos los enviaremos **directamente a todas las neuronas** de la primera capa oculta.

- Los **inputs** de la capa (outputs de la capa anterior) serán **una matriz**
- Otra **matriz** serán los **pesos de todas las neuronas** de la capa
- El **resultado** será **una matriz** con los **resultados de cada neurona**
- Aplicaremos la **misma función de activación a todas las neuronas de una capa**



Función de Activación
Aplicada a la matriz
resultado en conjunto

Por ejemplo: ReLU
Resultado = $\max(0, x)$

Resultado Neurona 4	Resultado Neurona 5
-3	6

Resultado Neurona 4	Resultado Neurona 5
0	6

Pruebas

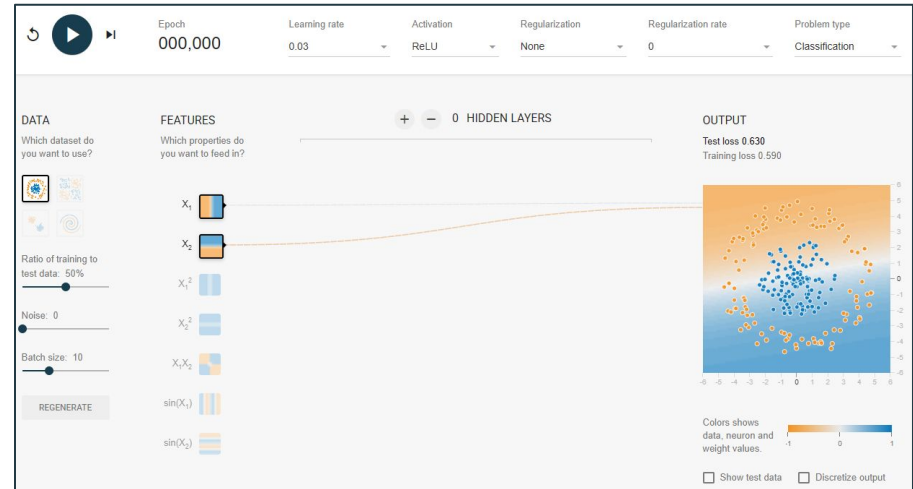
Una de las mejores formas para entender bien cómo pueden llegar a funcionar las redes neuronales es probando.

Os dejo aquí un enlace a una aplicación web abierta con la que podremos hacer pruebas:

[Aplicación](#)

También os dejo aquí un video donde juegan con esta misma aplicación:

[Explicación](#)



Práctica 2

Utilizando la herramienta online de TensorFlow realiza la siguiente actividad:

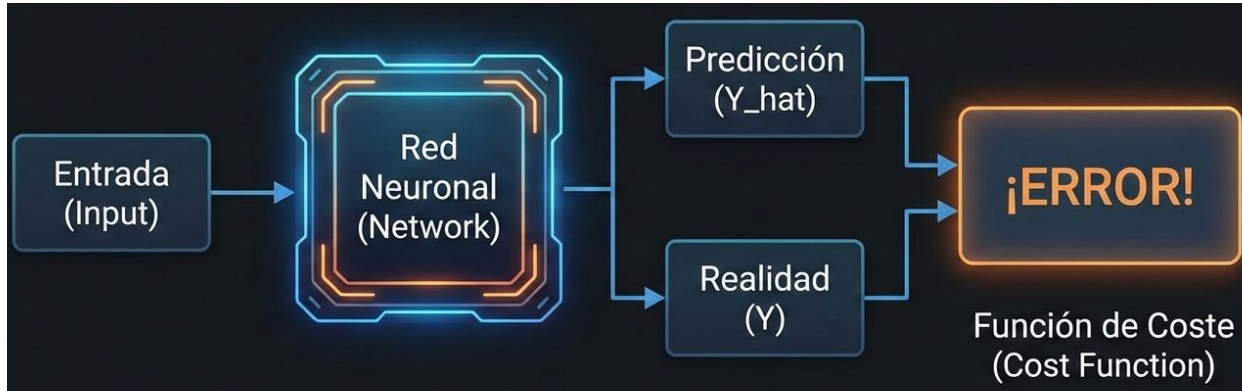
Práctica 2: Arquitectura y Topología de Redes Neuronales

La Función de Coste

Hasta ahora, nuestra red neuronal solo "opina": recibe datos y saca una predicción.

Para que aprenda, necesita saber **qué tan mal** lo ha hecho.

- **Definición:** La Función de Coste (Cost Function o Loss Function) es una fórmula matemática que cuantifica el error entre la predicción de la red y el valor real esperado.
- **El objetivo del juego:** El entrenamiento consiste exclusivamente en **minimizar** el valor de esta función. Si el coste es 0, la red es perfecta.



El Error Cuadrático Medio (ECM)

Es la función de coste estándar para problemas de regresión (predecir números).

Fórmula:

$$ECM = \frac{1}{N} \sum_{i=1}^N (Y_{real} - Y_{predicha})^2$$

¿Por qué elevamos al cuadrado?

1. **Elimina negativos:** El error siempre suma, nunca resta.
2. **Castiga los fallos grandes:** Un error de 2 vale 4, pero un error de 10 vale 100. Obliga a la red a no cometer errores graves.

Interpretación Geométrica del Coste

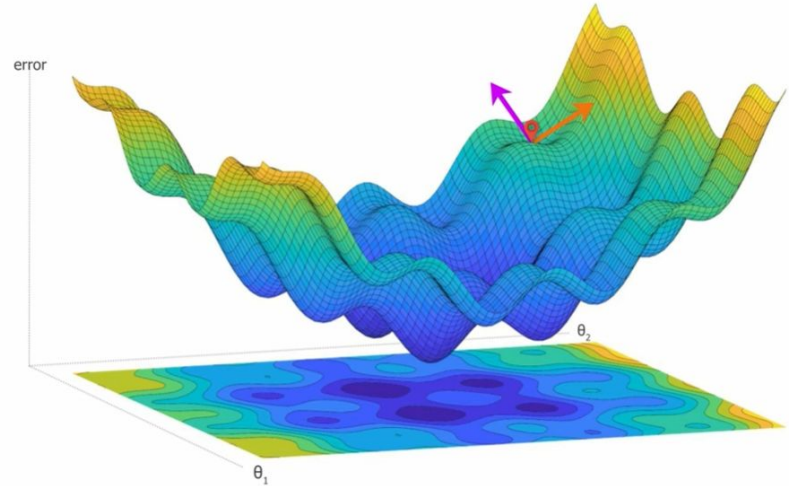
El valor del error depende de los **Pesos (W)** que tenga la red.

Si imaginamos todos los posibles valores de pesos, se genera un mapa 3D:

- **Ejes Horizontales:** Los valores de los pesos (w_1, w_2).
- **Altura (Eje Vertical):** El Error resultante.

Resultado: Un paisaje con montañas (malos pesos, mucho error) y valles (buenos pesos, poco error).

Misión: Encontrar las coordenadas (w_1, w_2) del punto más profundo del valle.



El Descenso del Gradiente

Estamos perdidos en la montaña del error con niebla (no vemos el mapa). Solo podemos sentir el suelo bajo nuestros pies.

El Algoritmo:

1. **Tantear (Gradiente):** Medimos la inclinación del suelo justo donde estamos.
2. **Decidir:** Si el suelo sube a la derecha, debemos ir a la izquierda.
3. **Avanzar:** Damos un paso en dirección contraria a la pendiente.

Matemáticamente: La "inclinación" es la **Derivada**. El algoritmo nos dice que siempre debemos restar la derivada a nuestros pesos.

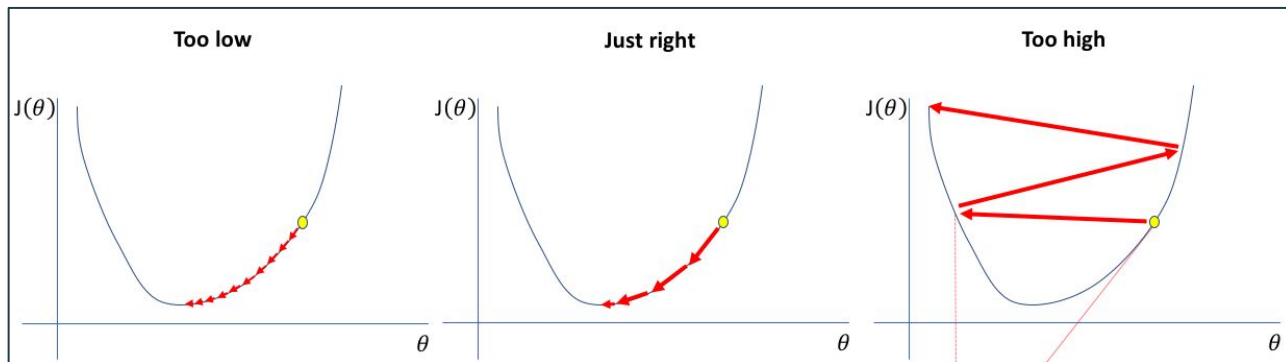
El tamaño del paso: Learning Rate (α)

Este parámetro decide cuánto avanzamos en cada paso de corrección.

La importancia de elegir bien:

- **Muy pequeño:** El descenso es lentísimo. Tardaremos "años" en entrenar.
- **Muy grande:** Damos saltos gigantes. Podemos saltarnos el valle y acabar en un sitio peor (**Divergencia**).

Valor típico: Suele ser un número pequeño, como 0.01 o 0.001.



Actualización de los Pesos

Una vez sabemos el error y la dirección, modificamos la red.

La Fórmula:

$$W_{nuevo} = W_{actual} - (\text{Learning Rate} \times \text{Gradiente})$$

¿Por qué restamos?

- Porque queremos ir en contra de la pendiente.
- Si la pendiente es positiva (subida), restamos para bajar.
- Si la pendiente es negativa (bajada), restar un negativo suma (avanzamos a la derecha).

Backpropagation: ¿Quién tuvo la culpa?

El Descenso del Gradiente nos dice *cómo* cambiar, pero el Backpropagation nos dice *qué* cambiar.

El Reto:

- Sabemos el Error Total al final de la red.
- Pero, ¿qué peso específico de la primera capa causó ese error?

La Solución: Deshacemos el camino de la red desde el final hasta el principio para repartir la "culpa" (el error) proporcionalmente entre las neuronas.

Desglosando el Gradiente (Regla de la Cadena)

Para llegar desde el Error final hasta un peso interno, atravesamos 3 eslabones.
Multiplicamos sus derivadas:

1. **Derivada del Error:** ¿Cuánto nos equivocamos en la salida?
2. **Derivada de la Activación:** ¿Cómo reacciona la neurona (Sigmoide/ReLU/Tanh)?
3. **Derivada de la Entrada:** ¿Cómo afecta el peso a la suma interna?

Fórmula:

Gradiente = Efecto Error $\times$ Sensibilidad Neurona $\times$ Entrada

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Matrices del Gradiente: ¿Por qué importa el valor de X?

La Matemática (Derivada de la suma):

- Dentro de la neurona, la operación es lineal: $z = (w \cdot x) + b$
- Si calculamos cómo cambia z al variar w (derivada parcial), la x actúa como una constante multiplicadora:

$$\frac{\partial z}{\partial w} = x$$

La Lógica (Sensibilidad):

- Esto significa que la "culpa" (gradiente) se amplifica por el valor de la entrada:
Gradiente Final = Error × Derivada Activación × x
- **Si $x \approx 0$ (Neurona apagada):** El gradiente es 0 → El peso **no cambia** (No aprende).
- **Si x es gigante (ej. 1.000.000):** El gradiente es enorme → El peso da un **salto brusco** (Inestabilidad).

La Recursividad (Hacia las capas ocultas)

Ya ajustamos la capa de salida. ¿Y las demás?

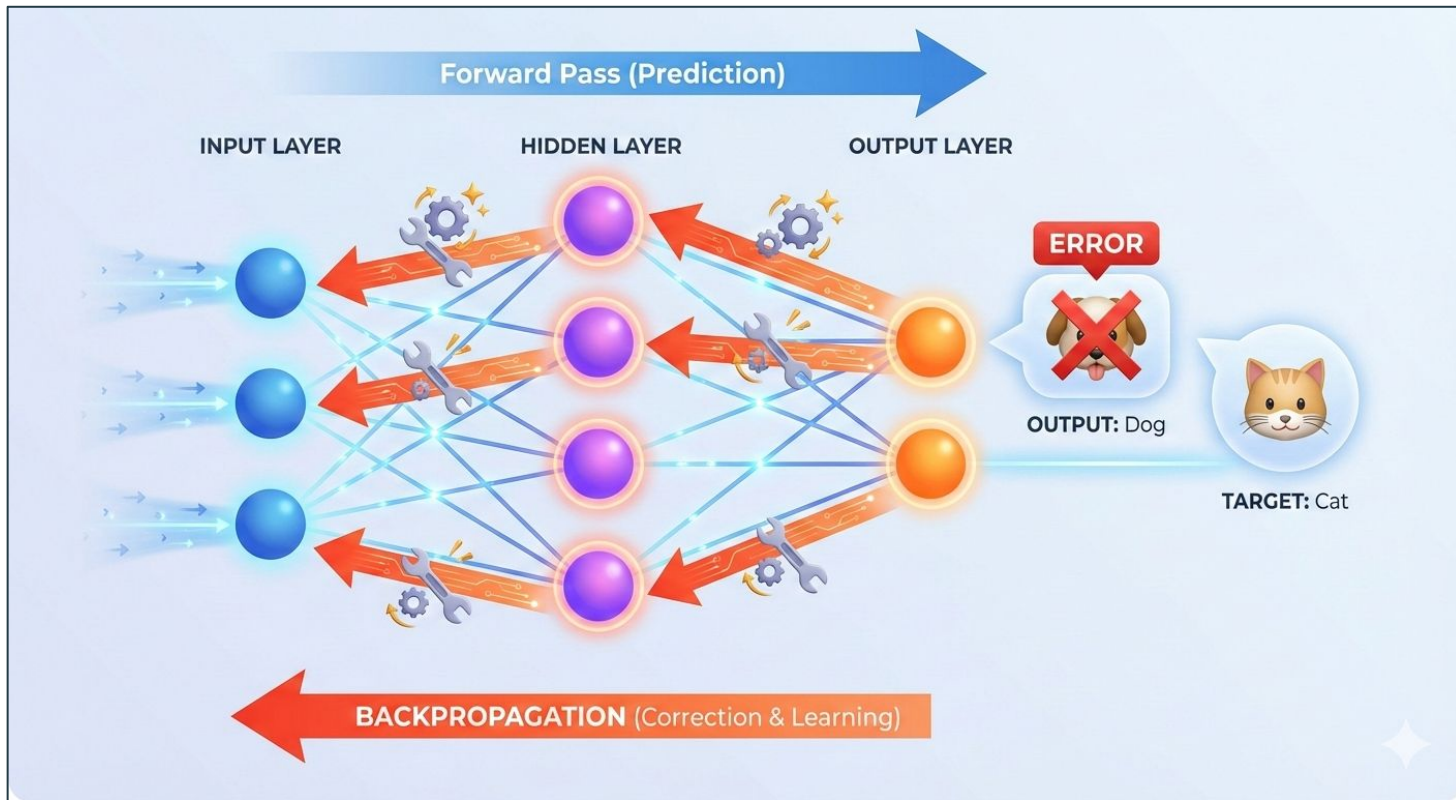
El error fluye hacia atrás:

- La "culpa" que llega a una neurona oculta es la suma de las culpas de todas las neuronas siguientes a las que está conectada.

Eficiencia:

- Calculamos la Capa 3 usando el error final.
- Calculamos la Capa 2 usando el error de la Capa 3.
- Calculamos la Capa 1 usando el error de la Capa 2.

Representación visual del Backpropagation



Resumen: El Bucle de Aprendizaje

El entrenamiento es repetir este ciclo miles de veces (Epochs):

1. **Forward Pass:** Los datos entran, la red predice.
2. **Compute Loss:** Comparamos predicción con realidad (ECM).
3. **Backward Pass:** Calculamos gradientes desde el final al principio (Backpropagation).
4. **Update Weights:** Actualizamos todos los pesos (Descenso del Gradiente).

Al terminar, la red ha aprendido.

El Objetivo del Aprendizaje: La Generalización

El objetivo fundamental de un modelo de Machine Learning no es minimizar el error en el conjunto de entrenamiento (E_{in}), sino minimizar el error esperado en datos futuros (E_{out}).

Capacidad de Generalización:

- Es la habilidad del modelo para adaptarse correctamente a datos nuevos, no observados previamente, basándose en los patrones aprendidos.

El Riesgo de la Alta Capacidad:

- Las Redes Neuronales Profundas son aproximadores universales de funciones. Tienen tal cantidad de parámetros que podrían teóricamente "memorizar" cada punto del dataset, incluyendo el ruido, perdiendo así su capacidad predictiva real.

Underfitting y Overfitting

El rendimiento del modelo se define por el compromiso entre Sesgo (Bias) y Varianza.

1. Underfitting (Subajuste) - Alto Sesgo:

- El modelo es demasiado simple para capturar la estructura subyacente de los datos.
- **Diagnóstico:** Alto error tanto en Entrenamiento como en Validación. El modelo no converge.

2. Overfitting (Sobreajuste) - Alta Varianza:

- El modelo es excesivamente complejo y se ajusta al **ruido estocástico** de los datos de entrenamiento en lugar de a la señal.
- **Diagnóstico:** Error de Entrenamiento muy bajo ($E_{in} \rightarrow 0$) pero Error de Validación alto y creciente.
- La red ha perdido generalidad; ha "memorizado" el ruido.

Monitorización de las Métricas de Pérdida

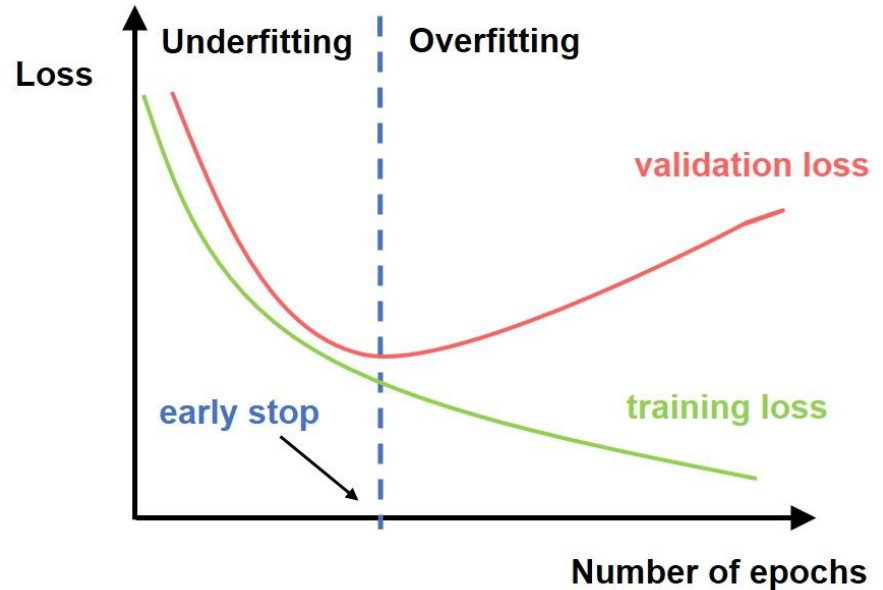
Para detectar el estado del entrenamiento, analizamos la evolución de la **Loss Function** a lo largo de las épocas (epochs).

Training Loss: Indica qué tan bien se ajusta el modelo a los datos conocidos. Tiende a disminuir asintóticamente.

Validation Loss: Indica la capacidad de generalización.

Punto de Divergencia:

- El momento crítico donde la *Validation Loss* deja de disminuir y comienza a aumentar, mientras la *Training Loss* sigue bajando.
- **Interpretación:** Este punto marca el inicio del **Overfitting**. A partir de aquí, el modelo está optimizando sobre ruido, no sobre patrones.



Estrategias para Mitigar el Overfitting

1. **Aumento de Datos (Data Augmentation):** Incrementar la diversidad del dataset de entrenamiento mediante transformaciones sintéticas, dificultando la memorización.
2. **Reducción de la Complejidad:** Disminuir el número de capas ocultas o neuronas para reducir la capacidad del espacio de hipótesis.
3. **Early Stopping:** Interrupción del entrenamiento en el momento exacto en que la métrica de validación deja de mejorar.
4. **Dropout (Regularización Estocástica):**
 - Técnica específica de Redes Neuronales. Durante el entrenamiento, se desactivan aleatoriamente neuronas con una probabilidad p .
 - Esto fuerza a la red a aprender características robustas y redundantes, evitando la co-adaptación de neuronas.